Highlights

A Householder-Based QR Factorization for Banded-Plus-Semiseparable Matrices in Linear Complexity

Tao Chen, Sheehan Olver

- We propose a fast QR factorization algorithm for banded-plus-semiseparable matrices using Householder transformations.
- The proposed algorithm achieves a strict linear computational complexity and effectively preserves the underlying matrix structures.
- A high-performance, open-source Julia package is developed and validated through extensive numerical benchmarks.

A Householder-Based QR Factorization for Banded-Plus-Semiseparable Matrices in Linear Complexity[⋆,⋆⋆]

Tao Chen[a,*], Sheehan Olver[a]

[a]Department of Mathematics, Imperial College London, South Kensington Campus, London, SW7 2AZ, United Kingdom

ARTICLE INFO

Keywords:
banded-plus-semiseparable matrices
QR factorization
linear complexity
structured matrices
direct solvers

ABSTRACT

We show that the QR factorization of a banded-plus-semiseparable (BPS) matrix is computable in optimal linear complexity with respect to the discretization size by showing that the intermediate stages of a QR factorization as computed using Householder reflection maintain a specific structure which has optimal storage. This theoretical result enables the design of stable, linear-complexity algorithms for solving the associated linear systems. Importantly, our algorithm outputs data in a standard LAPACK-compatible format. This allows users to switch to highly optimized dense matrix operations for the final solve, creating a fast hybrid approach. For symmetric BPS matrices, we further show that the reverse RQ product preserves the BPS structure, establishing the algebraic closure of this matrix class under orthogonal similarity transformations and facilitating efficient chain-structured matrix decompositions. Numerical experiments validate the optimal linear complexity, confirm high numerical accuracy, demonstrate excellent computational efficiency via LAPACK integration, and show substantial speedups compared with existing hierarchical approaches. The algorithms have been implemented in an open-source Julia package, providing an efficient and accessible platform for practical use.

Mention adaptivity, partial QR

1. Introduction

Banded-plus-semiseparable (BPS) matrices, expressible as

$$A = \underbrace{B}_{\text{banded}} + \underbrace{\operatorname{tril}(UV^\top, -1)}_{\text{lower semiseparable rank } r} + \underbrace{\operatorname{triu}(WS^\top, 1)}_{\text{upper semiseparable rank } p} \in \mathbb{R}^{n\times n},$$

where $U, V \in \mathbb{R}^{n\times r}$ and $W, S \in \mathbb{R}^{n,p}$ arise in numerous applications from spectral methods for differential equations Iserles (2025) to signal processing, control theory, and eigenvalue problems Vandebril, Van Barel and Mastronardi (2005b). Importantly, the inverse of a banded matrix is itself a BPS matrix, making this structure fundamental for representing inverses of banded matrices Rózsa, Bevilacqua, Romani and Favati (1991). Their structure requires only $O(n)$ storage as $n \to \infty$, which invites the development of $O(n)$ algorithms, a goal successfully achieved for iterations of the QR algorithm for symmetric semiseparable systems Vandebril, Van Barel and Mastronardi (2005a), and for solving linear systems with diagonal-plus-semiseparable matrices Eidelman and Gohberg (1997). However, generalizing these results to the case where the matrix is nonsymmetric, or the banded part B is a genuine banded matrix, rather than merely a diagonal one, and representing the QR factorization in its entirety presents significant algorithmic challenges.

Pioneering work has established stable $O(n)$ direct solvers for various classes of BPS and related rank-structured matrices. In particular, a mature and extensive literature has long leveraged sequences of Givens rotations to exploit these compact representations. The foundational theory and rotational ordering patterns for generic rank-structured systems are comprehensively detailed in the seminal books of Vandebril, Van Barel and Mastronardi (2008a) and Eidelman, Gohberg and Haimovici (2014), both of which provide exhaustive analyses of fast solvers, including QR and LU-type factorizations that adapt to BPS forms. Software implementations have historical precedents as well; most notably, the `SSPACK` library by Vandebril and Van Barel Vandebril and Van Barel delivers a robust Fortran framework for managing semiseparable operations via Givens transformations.

*Corresponding author
t.chen24@imperial.ac.uk (T. Chen); s.olver@imperial.ac.uk (.S. Olver)
ORCID(s):

Beyond these, alternative algorithmic frameworks include ULV factorizations Chandrasekaran and Gu (2003), hierarchically semiseparable (HSS) matrix techniques Chandrasekaran, Dewilde, Gu, Lyons and Pals (2007); Chandrasekaran, Gu and Pals (2006); Massei, Robol and Kressner (2020); Xia, Chandrasekaran, Gu and Li (2010), and rational Krylov approaches Fasino (2005); Vandebril, Van Barel and Mastronardi (2008b). Structure-preserving properties under rotational QR steps have also been extensively explored Delvaux and Van Barel (2006a,b); Eidelman, Gohberg and Olshevsky (2005); Delvaux and Van Barel (2008a); Vandebril et al. (2005b).

Explain existing Given's approaches start at bottom left and so aren't compatible with adaptivity. Mention that if look-up table is known ahead of time, and give example like $u = [1, 1/2, 1/3, \dots]$, then the complexity of partial QR does not depend on n but rather the number of columns upper-triangularised. Maybe reference Matt Colbrook/Townsends resolvant based eigenvalue stuff.

A special case of BPS matrices with no lower semiseparable part ($r = 0$) is almost banded matrices, which were used in Olver and Townsend (2013) to represent discretisations of differential equations using the ultraspherical spectral method, accompanied by an optimal complexity adaptive QR factorization and back-substitution solver.

Crucially, however, almost all existing $O(n)$ QR factorization schemes for BPS matrices in the rank-structured literature are fundamentally anchored in Givens rotations Delvaux and Van Barel (2008b); Mastronardi, Chandrasekaran and Van Huffel (2001); Van Camp, Mastronardi and Van Barel (2004). While rotational approaches are well-developed, they rely on localized transformations that do not yield the standard Householder factor matrix layout native to LAPACK storage formats and modern dense computing pipelines. A clear theoretical guarantee and an explicit proof showing that the standard QR factorization via Householder transformations preserves the compact BPS format has been missing from the literature.

In this paper, we close this theoretical and algorithmic gap. We prove that the QR factorization of a BPS matrix via Householder reflections yields a full factor matrix F (encoding both R and the reflectors) that is itself BPS. The core of our idea lies in characterizing the precise structural evolution during the reflection process. Although the BPS structure is not strictly invariant under individual Householder transformations, we demonstrate that the intermediate matrices remain within a well-defined space of structured perturbations, denoted as $\mathcal{P}(A)$ (Definition 4). The fundamental insight is that each Householder reflection preserves this extended structure (Lemma 4), ensuring that the final factor matrix F maintains a compact representation (Theorem 1). By leveraging this structural inheritance, we design a complete $O(n)$ direct solver—including $Q^\top$ application and backward substitution—implemented in a high-performance, open-source Julia package. Furthermore, we extend this framework to symmetric BPS matrices, proving that the RQ product also retains the BPS structure.

The rest of this paper is organized as follows. Section 2 discusses the factor matrix representation for the QR factorization as computed with Householder reflections, which proves a convenient language to discuss the perturbations of the intermediate stages of the factorization. Section 3 presents our main theoretical contributions on the preservation of a specific perturbation structure and the resulting BPS structure of the final QR factorization. Section 4 details the practical implementation of $O(n)$ algorithms for QR factorization, application of $Q^\top$, and backward substitution. Section 5 presents numerical experiments that confirm the linear complexity, verify the numerical stability, and demonstrate performance advantages of our algorithms, including their compatibility with LAPACK. We conclude in Section 6 with a discussion of future work. Finally, Appendix A extends this framework to symmetric BPS matrices, proving that the reverse RQ product also preserves the BPS format.

Menti
Comp
WY,
mentio
dense
DLAR
is $O(n$
for fix
block
k. Me
T is B
try to
figure
why.

Remark 1.1. *While this paper focuses on real square matrices, the techniques and results naturally extend to complex matrices by replacing transposes with conjugate transposes. The extension to rectangular matrices is also straightforward, as Householder QR factorization applies similarly to rectangular matrices, and the structural arguments carry over with minor adjustments. These extensions are omitted but represent immediate generalizations of the ideas presented here.*

Say that Q itself is BPS, and say how we can compute it. This is probably easy if we compute Compact WY since $Q = I + VTV^\top$. Thus $Q^{-1} = Q^T = I + VT^\top V^\top$

2. The Representation of the QR Factor Matrix

Consider the computation of the QR factorization using Householder reflections which we can write as:

$$A = \underbrace{(I - \tau_1 \mathbf{y}_1 \mathbf{y}_1^\top) \cdots (I - \tau_{n-1} \mathbf{y}_{n-1} \mathbf{y}_{n-1}^\top)}_{Q} \underbrace{\begin{bmatrix} r_{11} & r_{12} & r_{13} & \cdots & r_{1n} \\ & r_{22} & r_{23} & \cdots & r_{2n} \\ & & r_{33} & \cdots & r_{3n} \\ & & & \ddots & \vdots \\ & & & & r_{nn} \end{bmatrix}}_{R}$$

where

$$\mathbf{y}_k = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ 1 \\ y_{k+1,k} \\ \vdots \\ y_{n,k} \end{bmatrix} \begin{matrix} \left.\begin{matrix} \\ \\ \\ \end{matrix}\right\} k-1 \text{ zeros} \\ \left.\begin{matrix} \\ \\ \\ \end{matrix}\right\} n-k \text{ elements} \end{matrix} \tag{1}$$

and $\tau_k = 2\|\mathbf{y}_k\|^{-2}$ so that $\sqrt{\tau_k/2}\mathbf{y}_k$ has unit norm. Following the LAPACK format Anderson, Bai, Bischof, Blackford, Demmel, Dongarra, Du Croz, Greenbaum, Hammarling, McKenney and Sorensen (1999), the QR factorization can be represented by a *factor matrix* containing the entries of $\mathbf{y}_k$ and R:

$$F = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1k} \\ y_{21} & r_{22} & \cdots & r_{2k} \\ \vdots & \ddots & \ddots & \vdots \\ y_{k1} & \cdots & y_{k,k-1} & r_{kk} \end{bmatrix}$$

alongside a vector $\boldsymbol{\tau} = [\tau_1, \ldots, \tau_{n-1}, 0]^\top$.

We will demonstrate that F itself retains a BPS structure, which implies structure in R and the Householder reflections. Specifically, F has a lower semiseparable part of rank r, an upper semiseparable part of rank $r + p$, a lower bandwidth of ℓ, and an upper bandwidth of $\ell + m$.

Intermediate stages of the Householder reflection correspond to a *partial factor matrix*. In particular, if we apply k Householder reflections to partially upper triangularize A as

$$(I - \tau_k \mathbf{y}_k \mathbf{y}_k^\top) \cdots (I - \tau_1 \mathbf{y}_1 \mathbf{y}_1^\top) A = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1k} & r_{1,k+1} & \cdots & r_{1n} \\ & r_{22} & \cdots & r_{2k} & r_{2,k+1} & \cdots & r_{2n} \\ & & \ddots & \vdots & \vdots & \ddots & \vdots \\ & & & r_{kk} & r_{k,k+1} & \cdots & r_{kn} \\ & & & & a^k_{k+1,k+1} & \cdots & a^k_{k+1,n} \\ & & & & \vdots & \ddots & \vdots \\ & & & & a^k_{n,k+1} & \cdots & a^k_{nn} \end{bmatrix}.$$

We can encode this partial factorization in a partial factor matrix:

$$F_k = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1k} & r_{1,k+1} & \cdots & r_{1n} \\ y_{21} & r_{22} & \cdots & r_{2k} & r_{2,k+1} & \cdots & r_{2n} \\ \vdots & \ddots & \ddots & \vdots & \vdots & \ddots & \vdots \\ y_{k1} & \cdots & y_{k,k-1} & r_{kk} & r_{k,k+1} & \cdots & r_{kn} \\ y_{k+1,1} & \cdots & y_{k+1,k-1} & y_{k+1,k} & a^k_{k+1,k+1} & \cdots & a^k_{k+1,n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{n1} & \cdots & y_{n,k-1} & y_{nk} & a^k_{n,k+1} & \cdots & a^k_{nn} \end{bmatrix},$$

alongside $\boldsymbol{\tau}_k = [\tau_1, \ldots, \tau_k, 0, \ldots, 0]^\top$, for $k = 1, \ldots, n-1$.

Specifically, we define $F_0 := A$ as the initial state. Although the upper triangularization is computationally complete after $n-1$ steps, we formally define $F_n := F$ as the terminal state where the residual perturbation in the bottom-right 1×1 block of F_{n-1}—which at this stage is still implicitly represented within the structured perturbation format (to be explicitly defined in Section 3)—is formally absorbed into the final diagonal entry r_{nn}. Consequently, the upper triangular part of F_n constitutes the full factor R, while its strictly lower triangular part stores the Householder vectors $\{\mathbf{y}_k\}_{k=1}^{n-1}$.

The intermediate factor matrices *do not* have BPS structure, however, we shall show that it is a structured perturbation of BPS.

3. Structure of the factor matrix for BPS matrices

We now turn our attention to understanding the structure that F_k exhibits. In particular, we claim it is a structured perturbation of a BPS matrix. To be precise, we first introduce a notation for BPS matrices:

Definition 1. *Denote the set of banded matrices with lower-bandwidth ℓ, and upper-bandwidth m as* $\mathrm{BM}^{n\times n}_{(\ell,m)} \subset \mathbb{R}^{n\times n}$. *In particular, $B \in \mathrm{BM}^{n\times n}_{(\ell,m)} \subset \mathbb{R}^{n\times n}$ if $B = (b_{kj})_{k,j=1}^n \in \mathbb{R}^{n\times n}$ is a banded matrix satisfying $b_{kj} = 0$ for $k - j > \ell$ or $j - k > m$.*

Definition 2. *Denote the set of banded-plus-semiseparable matrices which have lower-semiseparable rank r, upper-semiseparable rank p, lower-bandwidth ℓ, and upper-bandwidth m as* $\mathrm{BPS}^{n\times n}_{(r,p),(\ell,m)} \subset \mathbb{R}^{n\times n}$. *In particular, $A \in \mathrm{BPS}^{n\times n}_{(r,p),(\ell,m)} \subset \mathbb{R}^{n\times n}$ if*

$$A = B + \mathrm{tril}(UV^\top, -1) + \mathrm{triu}(WS^\top, 1)$$

for $U, V \in \mathbb{R}^{n\times r}$, $W, S \in \mathbb{R}^{n\times p}$, and $B \in \mathrm{BM}^{n\times n}_{(\ell,m)}$.

We note that the BPS structure is not uniquely defined, so we implicitly assume that if we know $A \in \mathrm{BPS}^{n\times n}_{(r,p),(\ell,m)}$ we have access to U, V, W, S. This structure is preserved under principal subsection:

$$\begin{aligned} A[k:n, k:n] = B[k:n, k:n] &+ \mathrm{tril}(U[k:n,:]V[k:n,:]^\top, -1) \\ &+ \mathrm{triu}(W[k:n,:]S[k:n,:]^\top, 1) \in \mathrm{BPS}^{n-k+1\times n-k+1}_{(r,p),(\ell,m)}. \end{aligned}$$

We will also need to work with padded vectors or matrices:

Definition 3. *Denote a padded vector with only the first p rows being possibly nonzero as $\mathcal{S}_p^n \subset \mathbb{R}^n$ and a padded matrix with only the principal $p \times q$ subblock being possibly nonzero as $\mathcal{S}_{p\times q}^{m\times n} \subset \mathbb{R}^{m\times n}$. Explicitly, $\mathbf{x} \in \mathcal{S}_p^n$ if $\mathbf{x}[k] = 0$ unless $k \le p$ and $A \in \mathcal{S}_{p\times q}^{m\times n}$ if $A[k, j] = 0$ unless $k \le p$ and $j \le q$.*

Note for example if $B \in \mathrm{BM}_{(\ell,m)}^{n\times n}$ then $B[k : n, k] \in \mathcal{S}_{\ell+1}^{n-k+1}$ and $B[k, k : n] \in \mathcal{S}_{m+1}^{n-k+1}$.

We will now describe the structure of each portion of the partial factor matrix. We will describe the bottom right in terms of the following linear space of matrices:

Definition 4. *Given*

$$A = B + \mathrm{tril}(UV^\top, -1) + \mathrm{triu}(WS^\top, 1) \in \mathrm{BPS}_{(r,p),(\ell,m)}^{n\times n}$$

define the linear space:

$$\begin{aligned}\mathcal{P}(A) := \Big\{ & UJS^\top + UKU^\top A + UE + XS^\top + YU^\top A + Z : \\ & J \in \mathbb{R}^{r\times p}, K \in \mathbb{R}^{r\times r}, E \in \mathcal{S}_{r\times(\ell+m)}^{r\times n}, X \in \mathcal{S}_{\ell\times p}^{n\times p}, \\ & Y \in \mathcal{S}_{\ell\times r}^{n\times r}, Z \in \mathcal{S}_{\ell\times(\ell+m)}^{n\times n} \Big\} \subset \mathbb{R}^{n\times n}.\end{aligned}$$

For convenience we use subscript s to define the nonzero portions of these matrices as $E_s := E[:, 1 : \min(\ell+m, n)]$, $X_s := X[1 : \min(\ell, n), :]$, $Y_s := Y[1 : \min(\ell, n), :]$, and $Z_s := Z[1 : \min(\ell, n), 1 : \min(\ell+m, n)]$.

We will show that F_k has the following structure: The first k rows/columns of F_k will maintain the BPS structure albeit with increased upper semiseparable rank of $p + r$ and upper bandwidth of $\ell + m$. The bottom right will be a structured perturbation of a BPS matrix.

In particular, the partial factor matrices will satisfy the following:

Definition 5. *We say $F_k \in \mathrm{BPSF}_k(A)$ if for*

$$\tilde{S} := \begin{bmatrix} S & A^\top U \end{bmatrix} \in \mathbb{R}^{n\times(r+p)},$$

there exist $B_k \in \mathrm{BM}_{(\ell,\ell+m)}^{n\times n}$, $V_k \in \mathbb{R}^{n\times r}$, $W_k \in \mathbb{R}^{n\times(r+p)}$, so that the first k rows and columns satisfy the conditions of the BPS format:

$$\begin{aligned} F_k[:, 1 : k] &= (B_k + \mathrm{tril}(UV_k^\top, -1) + \mathrm{triu}(W_k\tilde{S}^\top, 1))[:, 1 : k] \\ F_k[1 : k, :] &= (B_k + \mathrm{tril}(UV_k^\top, -1) + \mathrm{triu}(W_k\tilde{S}^\top, 1))[1 : k, :] \end{aligned}$$

whilst the bottom right block is a perturbed BPS matrix:

$$F_k[k+1 : n, k+1 : n] = A[k + 1 : n, k + 1 : n] + P_k,$$

where $P_k \in \mathcal{P}(A[k + 1 : n, k + 1 : n])$.

We will show that the factor matrix $F_k \in \mathrm{BPSF}_k(A)$ by induction. As a preliminary step we note that the perturbation structure of the bottom right is preserved under truncation:

Lemma 1. *If $P \in \mathcal{P}(A)$ then $P[k : n, k : n] \in \mathcal{P}(A[k : n, k : n])$.*

Proof. We show for $k = 2$ with the other cases following by induction. We write:

$$\begin{aligned} P[2 : n, 2 : n] = U[2 : n, :]JS[2 : n, :]^\top + U[2 : n, :]K & \underbrace{U^\top A[:, 2 : n]}_{= U[2:n,:]^\top A[2:n,2:n] + U[1,:]A[1,2:n]^\top} \\ + U[2 : n, :]E[:, 2 : n] + X[2 : n, :]S[2 : n, :]^\top & \end{aligned}$$

$$+ Y[2:n,:]U^\top A[:,2:n] + Z[2:n,2:n].$$

We can then write

$$U[2:n,:]KU[1,:]A[1,2:n]^\top = U[2:n,:]\underbrace{KU[1,:]B[1,2:n]^\top}_{\in S^{r\times(n-1)}_{r\times m}} + U[2:n,:]\underbrace{KU[1,:]W[1,:]^\top}_{\in\mathbb{R}^{r\times p}} S[2:n,:]^\top.$$

Similarly,

$$Y[2:n,:]U[1,:]A[1,2:n]^\top = \underbrace{Y[2:n,:]U[1,:]B[1,2:n]^\top}_{\in S^{(n-1)\times(n-1)}_{(\ell-1)\times m}} + \underbrace{Y[2:n,:]U[1,:]W[1,:]^\top}_{\in S^{(n-1)\times p}_{(\ell-1)\times p}} S[2:n,:]^\top.$$

The lemma follows since we have

$$\begin{aligned}P[2:n,2:n] = {} & U[2:n,:](J + KU[1,:]W[1,:]^\top)S[2:n,:]^\top + \\ & U[2:n,:]KU[2:n,:]^\top A[2:n,2:n] \\ & + U[2:n,:](E[:,2:n] + KU[1,:]B[1,2:n]^\top) \\ & + (X[2:n,:] + Y[2:n,:]U[1,:]W[1,:]^\top)S[2:n,:]^\top \\ & + Y[2:n,:]U[2:n,:]^\top A[2:n,2:n] \\ & + Z[2:n,2:n] + Y[2:n,:]U[1,:]B[1,2:n]^\top,\end{aligned}$$

which is precisely the required form. □

The next step is to understand the structure of the Householder reflector, which corresponds to rows $k+1$ through n of the k-th column of F_k:

Lemma 2. *For $1 \le k \le n-1$, suppose $F_{k-1} \in \mathrm{BPSF}_{k-1}(A)$. Then there exists B_k and V_k such that*

$$F_k[:,1:k] = (B_k + \mathrm{tril}(UV_k^\top, -1) + \mathrm{triu}(W_{k-1}\tilde{S}^\top, 1))[:,1:k]$$

Proof. The first $k-1$ columns follow by defining:

$$\begin{aligned}B_k[:,1:k-1] &:= B_{k-1}[:,1:k-1] \\ V_k[1:k-1,:] &:= V_{k-1}[1:k-1,:].\end{aligned}$$

The first k rows of the k-th column follow by defining $B_k[k,k] := r_{kk}$. Thus the main contribution of this lemma is structure in the k-th Householder reflector which corresponds to the $k+1$ through nth rows.

Since $F_{k-1} \in \mathrm{BPSF}_{k-1}(A)$ we may write

$$F_{k-1}[k:n,k:n] = A[k:n,k:n] + P_{k-1}$$

where

$$\begin{aligned}P_{k-1} := {} & U[k:n,:]J_kS[k:n,:]^\top + U[k:n,:]K_kU[k:n,:]^\top A[k:n,k:n] \\ & + U[k:n,:]E_k + X_kS[k:n,:]^\top + Y_kU[k:n,:]^\top A[k:n,k:n] + Z_k\end{aligned}$$

with $J_k \in \mathbb{R}^{r\times p}$, $K_k \in \mathbb{R}^{r\times r}$, $E_k \in S^{r\times(n-k+1)}_{r\times(\ell+m)}$, $X_k \in S^{(n-k+1)\times p}_{\ell\times p}$, $Y_k \in S^{(n-k+1)\times r}_{\ell\times r}$, and $Z_k \in S^{(n-k+1)\times(n-k+1)}_{\ell\times(\ell+m)}$.

The k-th Householder reflection is defined in terms of

$$\begin{aligned} F_{k-1}[k:n,k] &= A[k:n,k] + P_{k-1}[1:n-k+1,1] \\ &= B[k:n,k] + \begin{bmatrix} 0 \\ U[k+1:n,:] \end{bmatrix} V[k,:] \\ &\quad + \begin{bmatrix} 0 \\ U[k+1:n,:] \end{bmatrix} \underbrace{(J_k S[k,:] + K_k U^\top A[:,k] + E_k[:,1])}_{=:\mathbf{v}_k \in \mathbb{R}^r} \\ &\quad + \underbrace{\begin{bmatrix} X_{ks} S[1,:] + Y_{ks} U^\top A[:,k] + Z_{ks}[:,1] \in \mathbb{R}^{\min(\ell, n-k+1)} \\ \mathbf{0} \end{bmatrix}}_{=:\tilde{\mathbf{b}}_k}. \end{aligned} \tag{2}$$

Letting

$$\alpha_k := \operatorname{sign}(F_{k-1}[k,k]) \left| F_{k-1}[k,k] + \operatorname{sign}(F_{k-1}[k,k]) \| F_{k-1}[k:n,k] \| \right|, \tag{3}$$

we define:

$$B_k[k+1:n,k] := \frac{B[k+1:n,k] + \tilde{\mathbf{b}}_k[2:n-k+1]}{\alpha_k} \tag{4}$$

and

$$V_k[k,:] := \frac{\mathbf{v}_k + V[k,:]}{\alpha_k}. \tag{5}$$

The remaining (not structurally zero) entries of B_k and V_k are at this point arbitrary. □

We now show the first k rows of F_k (corresponding to R) also have the specified form:

Lemma 3. *For $1 \le k \le n-1$, suppose $F_{k-1} \in \mathrm{BPSF}_{k-1}(A)$. Then there exists B_k, V_k, W_k such that*

$$F_k[1:k,:] = (B_k + \operatorname{tril}(UV_k^\top, -1) + \operatorname{triu}(W_k \tilde{S}^\top, 1))[1:k,:]$$

Proof. The first $k-1$ rows follow by defining V_k as before and:

$$\begin{aligned} B_k[1:k-1,:] &:= B_{k-1}[1:k-1,:], \\ W_k[1:k-1,:] &:= W_{k-1}[1:k-1,:]. \end{aligned}$$

We now consider the first row in applying the k-th Householder reflection to

$$\begin{aligned} A_{k-1} &:= F_{k-1}[k:n,k:n] \\ &= A[k:n,k:n] + \underbrace{(U[k:n,:]J_k + X_k[1:n-k+1,:])}_{=:\hat{T}_1 \in \mathbb{R}^{(n-k+1)\times p}} S[k:n,:]^\top \\ &\quad + \underbrace{(U[k:n,:]K_k + Y_k[1:n-k+1,:])}_{=:T_2 \in \mathbb{R}^{(n-k+1)\times r}} U[k:n,:]^\top A[k:n,k:n] \\ &\quad + \underbrace{U[k:n,:]E_k[:,1:n-k+1] + Z_k[1:n-k+1,1:n-k+1]}_{=:\hat{T}_3 \in S^{(n-k+1)\times(n-k+1)}_{(n-k+1)\times(\ell+m)}}. \end{aligned}$$

Since

$$U[k:n,:]^\top A[k:n,k:n] = (A^\top U)[k\,:\,n,:]^\top - \underbrace{(\sum_{t=1}^{k-1} U[t,:]W[t,:]^\top)}_{=:P_1\in\mathbb{R}^{r\times p}} S[k\,:\,n,:]^\top - \underbrace{\sum_{t=\max(k-m,1)}^{k-1} U[t,:]B[t,k\,:\,n]}_{=:P_2\in S^{r\times(n-k+1)}_{r\times m}}, \tag{6}$$

we can also write A_{k-1} as

$$A_{k-1} = A[k:n,k:n] + \underbrace{(\hat{T}_1 - T_2P_1)}_{=:T_1\in\mathbb{R}^{(n-k+1)\times p}} S[k:n,:]^\top + T_2(U^\top A)[:,k:n] + \underbrace{\hat{T}_3 - T_2P_2}_{=:T_3\in S^{(n-k+1)\times(n-k+1)}_{(n-k+1)\times(\ell+m)}}.$$

Noting that

$$A[k,k:n]^\top = B[k,k:n]^\top + \underbrace{\mathbf{e}_k^\top \operatorname{triu}(WS^\top,1)[:,k:n]}_{W[k,:]^\top S[k:n,:]^\top - W[k,:]^\top S[k,:]\mathbf{e}_1^\top},$$

we can write the first row as

$$\mathbf{e}_1^\top A_{k-1} = \mathbf{s}_1^\top + \underbrace{(W[k,:]^\top + \mathbf{e}_1^\top T_1)}_{=:\mathbf{w}_1^\top\in\mathbb{R}^{1\times p}} S[k:n,:]^\top + \underbrace{\mathbf{e}_1^\top T_2}_{=:\tilde{\mathbf{w}}_1^\top\in\mathbb{R}^{1\times r}} (A^\top U)[k:n,:]^\top$$

for the sparse row vector (which will correspond to a slice of a banded matrix)

$$\mathbf{s}_1^\top := \underbrace{B[k,k:n]^\top}_{\in S^{1\times(n-k+1)}_{1\times(m+1)}} - \underbrace{W[k,:]^\top S[k,:]\mathbf{e}_1^\top}_{\in S^{1\times(n-k+1)}_{1\times 1}} + \underbrace{\mathbf{e}_1^\top T_3}_{\in S^{1\times(n-k+1)}_{1\times(\ell+m)}} \in S^{1\times(n-k+1)}_{1\times(\ell+m+1)}.$$

For the vector associated with the k-th Householder reflection

$$\mathbf{y}_k := \begin{bmatrix} 1 \\ F_k[k+1:n,k] \end{bmatrix} = \underbrace{B_k[k:n,k] + (1 - B_k[k,k] - U[k,:]^\top V_k[k,:])\mathbf{e}_1}_{=:\hat{\mathbf{b}}_k\in S^{n-k+1}_{\ell+1}} + U[k:n,:]V_k[k,:], \tag{7}$$

we first write

$$\mathbf{y}_k^\top A[k:n,k:n] = \mathbf{s}_2^\top + V_k[k,:]^\top U[k:n,:]^\top A[k:n,k:n] + \hat{\mathbf{b}}_k^\top W[k:n,:]S[k:n,:]^\top \tag{8}$$

for

$$\mathbf{s}_2^\top := \underbrace{\hat{\mathbf{b}}_k^\top B[k:n,k:n]}_{\in S^{1\times(n-k+1)}_{1\times(\ell+m+1)}} + \underbrace{\hat{\mathbf{b}}_k^\top \operatorname{tril}(U[k:n,:]V[k:n,:]^\top,-1)}_{\in S^{1\times(n-k+1)}_{1\times\ell+1}}$$

$$-\underbrace{\hat{\mathbf{b}}_k^\top \operatorname{tril}(W[k:n,:]S[k:n,:]^\top,0)}_{\in S_{1\times(\ell+1)}^{1\times(n-k+1)}} \in S_{1\times(\ell+m+1)}^{1\times(n-k+1)}.$$

We therefore find

$$\begin{aligned}\mathbf{y}_k^\top A_{k-1} = &\underbrace{\mathbf{s}_2^\top - V_k[k,:]^\top P_2 + \mathbf{y}_k^\top T_3}_{=:\mathbf{s}_3^\top \in S_{1\times\ell+m+1}^{1\times(n-k+1)}} + \underbrace{(V_k[k,:]^\top + \mathbf{y}_k^\top T_2)}_{=:\tilde{\mathbf{w}}_2^\top \in \mathbb{R}^{1\times r}}(A^\top U)[k:n,:]^\top \\ &+ \underbrace{(\hat{\mathbf{b}}_k^\top W[k:n,:] - V_k[k,:]^\top P_1 + \mathbf{y}_k^\top T_1)}_{=:\mathbf{w}_2^\top \in \mathbb{R}^{1\times p}} S[k:n,:]^\top.\end{aligned}$$

Thus we find for $\beta_k := -\tau_k \mathbf{e}_1^\top \mathbf{y}_k$ that applying the Householder reflection yields

$$\begin{aligned}\mathbf{e}_1^\top (I - \tau_k \mathbf{y}_k \mathbf{y}_k^\top) A_{k-1} = &\mathbf{s}_1^\top + \beta_k \mathbf{s}_3^\top + (\mathbf{w}_1^\top + \beta_k \mathbf{w}_2^\top) S[k:n,:]^\top \\ &+ (\tilde{\mathbf{w}}_1^\top + \beta_k \tilde{\mathbf{w}}_2^\top)(A^\top U)[k:n,:]^\top\end{aligned}$$

and hence the results from defining

$$W_k[k,:]^\top := \begin{bmatrix}(\mathbf{w}_1^\top + \beta_k \mathbf{w}_2^\top) & (\tilde{\mathbf{w}}_1^\top + \beta_k \tilde{\mathbf{w}}_2^\top)\end{bmatrix} \in \mathbb{R}^{1\times(r+p)} \tag{9}$$

and

$$B_k[k,k:n]^\top := \mathbf{s}_1^\top + \beta_k \mathbf{s}_3^\top \in S_{1\times(\ell+m+1)}^{1\times(n-k+1)}. \tag{10}$$

□

We put these results together to show that the structure for partial factor matrices is preserved at each iteration:

Lemma 4. *For $1 \le k \le n-1$, suppose $F_{k-1} \in \mathrm{BPSF}_{k-1}(A)$. Then $F_k \in \mathrm{BPSF}_k(A)$.*

Proof. We have already showed the result apart from the bottom right, in particular we need to show

$$\begin{aligned}A_k &:= F_k[(k+1):n,(k+1):n] \\ &= ((I - \tau_k \mathbf{y}_k \mathbf{y}_k^\top) A_{k-1})[2:n-k+1, 2:n-k+1] \\ &\in A[(k+1):n,(k+1):n] + \mathcal{P}(A[(k+1):n,(k+1):n]).\end{aligned}$$

From Lemma 1 we know that since $A_{k-1} \in A[k:n,k:n] + \mathcal{P}(A[k:n,k:n])$ we have

$$A_{k-1}[2:(n-k+1),2:(n-k+1)] \in A[(k+1):n,(k+1):n] + \mathcal{P}(A[(k+1):n,(k+1):n]).$$

By applying the same technique as in the proof of Lemma 1, we can express

$$\begin{aligned}&A_{k-1}[2:(n-k+1),2:(n-k+1)] \\ =&A[(k+1):n,(k+1):n] + U[(k+1):n,:]J^{(k)}S[(k+1):n,:]^\top \\ &+ U[(k+1):n,:]K^{(k)}U[(k+1):n,:]^\top A[(k+1):n,(k+1):n] \\ &+ U[(k+1):n,:]E^{(k)} + X^{(k)}S[(k+1):n,:]^\top \\ &+ Y^{(k)}U[(k+1):n,:]^\top A[(k+1):n,(k+1):n] + Z^{(k)}\end{aligned}$$

with $J^{(k)} \in \mathbb{R}^{r\times p}$, $K^{(k)} \in \mathbb{R}^{r\times r}$, $E^{(k)} \in S_{r\times(\ell+m)}^{r\times(n-k)}$, $X^{(k)} \in S_{\ell\times p}^{(n-k)\times p}$, $Y^{(k)} \in S_{\ell\times r}^{(n-k)\times r}$ and $Z^{(k)} \in S_{\ell\times(k+m)}^{(n-k)\times(n-k)}$.

We thus need only consider $\mathbf{y}_k \mathbf{y}_k^\top A_{k-1}$. Let $\hat{U}_k \in \mathbb{R}^{n-k+1}$ s.t. $\hat{U}_k[1,:] = \mathbf{0}$ and $\hat{U}_k[2:n-k+1,:] = U[k+1:n,:]$; $\hat{A}_k \in \mathbb{R}^{(n-k+1)\times(n-k+1)}$ s.t. $\hat{A}_k[1,:] = \mathbf{0}$ and $\hat{A}_k[2:n-k+1,:] = A[k+1:n,:]$.

Using the technique from the proof of Lemma 3 we can write

$$U[k:n,:]^\top A[k:n,k:n] = \hat{U}_k^\top \hat{A}_k + \underbrace{U[k,:]W[k,:]^\top}_{=:P_3\in\mathbb{R}^{r\times p}} S[k:n,:]^\top + \underbrace{U[k,:]B[k,k:n] - U[k,:]W[k,:]^\top S[k,:]\mathbf{e}_1^\top}_{=:P_4\in S^{r\times(n-k+1)}_{r\times m}}$$

and

$$\mathbf{y}_k^\top A_{k-1} = \underbrace{\mathbf{s}_2^\top + V_k[k,:]^\top P_4 + \mathbf{y}_k^\top T_3}_{=:\mathbf{s}_4^\top\in S^{1\times(n-k+1)}_{1\times\ell+m+1}} + \underbrace{(V_k[k,:]^\top + \mathbf{y}_k^\top T_2)}_{=:\tilde{\mathbf{w}}_2^\top\in\mathbb{R}^{1\times r}} \hat{U}_k^\top \hat{A}_k + \underbrace{(\hat{\mathbf{b}}_k^\top W[k:n,:] + V_k[k,:]^\top P_3 + \mathbf{y}_k^\top T_1)}_{=:\mathbf{w}_3^\top\in\mathbb{R}^{1\times p}} S[k:n,:]^\top.$$

Therefore,

$$\begin{aligned}\mathbf{y}_k\mathbf{y}_k^\top A_{k-1} &= (\hat{\mathbf{b}}_k + U[k:n,:]V_k[k,:])\times \\ &\quad (\mathbf{s}_4^\top + \tilde{\mathbf{w}}_2^\top \hat{U}_k^\top \hat{A}_k + \mathbf{w}_3^\top S[k:n,:]^\top) \\ &= \underbrace{\hat{\mathbf{b}}_k\mathbf{s}_4^\top}_{\in S^{(n-k+1)\times(n-k+1)}_{(\ell+1)\times(\ell+m+1)}} + \underbrace{\hat{\mathbf{b}}_k\tilde{\mathbf{w}}_2^\top}_{\in S^{(n-k+1)\times r}_{(\ell+1)\times r}} \hat{U}_k^\top\hat{A}_k + \underbrace{\hat{\mathbf{b}}_k\mathbf{w}_3^\top}_{\in S^{(n-k+1)\times p}_{(\ell+1)\times p}} S[k:n,:]^\top \\ &\quad + U[k:n,:]\underbrace{V_k[k,:]\mathbf{s}_4^\top}_{\in S^{r\times(n-k+1)}_{r\times(\ell+m+1)}} + U[k:n,:]\underbrace{V_k[k,:]\tilde{\mathbf{w}}_2^\top}_{\in\mathbb{R}^{r\times r}}\hat{U}_k^\top\hat{A}_k \\ &\quad + U[k:n,:]\underbrace{V_k[k,:]\mathbf{w}_3^\top}_{\in\mathbb{R}^{r\times p}} S[k:n,:]^\top.\end{aligned}$$

Noticing that $(\hat{U}_k^\top\hat{A}_k)[:,2:n-k+1] = U[k+1:n,:]^\top A[k+1:n,k+1:n]$, we can further write

$$\begin{aligned}A_k =& A_{k-1}[2:n-k+1,2:n-k+1] - (\tau_k\mathbf{y}_k\mathbf{y}_k^\top A_{k-1})[2:n-k+1,2:n-k+1] \\ =& A[(k+1):n,(k+1):n] + U[(k+1):n,:]J_{k+1}S[(k+1):n,:]^\top \\ &+ U[(k+1):n,:]K_{k+1}U[(k+1):n,:]^\top A[(k+1):n,(k+1):n] \\ &+ U[(k+1):n,:]E_{k+1} + X_{k+1}S[(k+1):n,:]^\top \\ &+ Y_{k+1}U[(k+1):n,:]^\top A[(k+1):n,(k+1):n] + Z_{k+1} \\ \in& A[(k+1):n,(k+1):n] + \mathcal{P}(A[(k+1):n,(k+1):n])\end{aligned}$$

where

$$J_{k+1} := J^{(k)} - \tau_k V_k[k,:]\mathbf{w}_3^\top \in \mathbb{R}^{r\times p}, \tag{11}$$

$$K_{k+1} := K^{(k)} - \tau_k V_k[k,:]\tilde{\mathbf{w}}_2^\top \in \mathbb{R}^{r\times r}, \tag{12}$$

$$E_{k+1} := E^{(k)} - \tau_k V_k[k,:]\mathbf{s}_4^\top[2:n-k+1] \in S^{r\times(n-k)}_{r\times(\ell+m)}, \tag{13}$$

$$X_{k+1} := X^{(k)} - \tau_k\hat{\mathbf{b}}_k[2:n-k+1]\mathbf{w}_3^\top \in S^{(n-k)\times p}_{\ell\times p}, \tag{14}$$

$$Y_{k+1} := Y^{(k)} - \tau_k \hat{\mathbf{b}}_k[2:n-k+1]\tilde{\mathbf{w}}_2^\top \in \mathcal{S}_{\ell\times r}^{(n-k)\times r}, \tag{15}$$

$$Z_{k+1} := Z^{(k)} - \tau_k \hat{\mathbf{b}}_k[2:n-k+1]\mathbf{s}_4^\top[2:n-k+1] \in \mathcal{S}_{\ell\times(\ell+m)}^{(n-k)\times(n-k)}. \tag{16}$$

□

We now arrive at the main result: the final factor matrix is BPS:

Theorem 1. *The QR factorization of a BPS matrix yields a BPS factor matrix:* $F_n \in \mathrm{BPS}^{n\times n}_{(r,r+p),(\ell,\ell+m)}$.

Proof. It is easy to see that $F_0 := A \in \mathrm{BPSF}_0(A)$ (with the parameter matrices J, K, E, X, Y and Z all $\mathbf{0}$). From Lemma 2, 3, and 4, an induction argument shows that F_{n-1} can be written as

$$F_{n-1} = B_{n-1} + \mathrm{tril}(UV_{n-1}^\top, -1) + \mathrm{triu}(W_{n-1}\tilde{S}^\top, 1)$$

except at the entry $F_n[n,n]$.

Now define $V_n := V_{n-1}$, $W_n := W_{n-1}$, and let B_n coincide with B_{n-1} everywhere except for the last diagonal entry $B_n[n,n] = F_{n-1}[n,n]$. We then obain

$$F_n = B_n + \mathrm{tril}(UV_n^\top, -1) + \mathrm{triu}(W_n\tilde{S}^\top, 1).$$

□

4. Fast Algorithms for QR Factorization and Direct Solvers

We now discuss the practical implementation of an $O(n)$ algorithm for computing the QR factorization of a BPS matrix based on theorem 1.

Algorithm 4.0.1 Fast QR Factorization for BPS Matrices

Input Banded-plus-semiseparable matrix $A \in \mathrm{BPS}^{n\times n}_{(r,p),(\ell,m)}$.
Output Factor matrix $F \in \mathrm{BPS}^{n\times n}_{(r,r+p),(\ell,\ell+m)}$ and scaling vector $\boldsymbol{\tau} \in \mathbb{R}^n$.
Initialization:
Compute $A^\top U$ in $O(n)$ Chandrasekaran, Dewilde, Gu, Pals and van der Veen (2002) and set $\tilde{S} \leftarrow \begin{bmatrix} S & A^\top U \end{bmatrix}$
Initialize $B_n \leftarrow \mathbf{0}_{n\times n}$, $V_n \leftarrow \mathbf{0}_{n\times r}$, $W_n \leftarrow \mathbf{0}_{n\times(r+p)}$
Initialize parameter matrices: $J \leftarrow \mathbf{0}_{r\times p}$, $K \leftarrow \mathbf{0}_{r\times r}$, $E_s \leftarrow \mathbf{0}_{r\times\min(\ell+m,n)}$, $X_s \leftarrow \mathbf{0}_{\min(\ell,n)\times p}$, $Y_s \leftarrow \mathbf{0}_{\min(\ell,n)\times r}$, $Z_s \leftarrow \mathbf{0}_{\min(\ell,n)\times\min(\ell+m,n)}$
for $k = 1$ to $n-1$ **do**
  Step 1: Form Householder vector $\mathbf{y}_k$
  Run FormHouseholderVector($A_{k-1}, U[k:n,:], V[k:n,:], \ell$) (Algorithm 4.0.2) to get:
  $\bar{\mathbf{k}}_k \leftarrow$ low-rank part from $\mathbf{y}_k$
  $\mathbf{b}_k \leftarrow$ banded part from $\mathbf{y}_k$
  $\tau_k \leftarrow$ scaling coefficient
  $\boldsymbol{\tau}[k] \leftarrow \tau_k$
  Step 2: Store k-th column of F
  $V_n[k,:] \leftarrow \bar{\mathbf{k}}_k$ (Store low-rank generator)
  for $j = k+1$ to $\min(k+\ell, n)$ **do**
    $B_n[j,k] \leftarrow \mathbf{b}_k[j-k+1]$ (Store banded part)
  end for
  Step 3: Compute and store k-th row of F
  $[\tilde{\mathbf{w}}_k, \tilde{\mathbf{d}}_k] \leftarrow$ ComputeRowUpdate($A_{k-1}, U, \tilde{S}, \mathbf{k}_k, \mathbf{b}_k, \tau_k$) (Algorithm 4.0.3)
  $W_n[k,:] \leftarrow \tilde{\mathbf{w}}_k$
  for $j = k$ to $\min(k+\ell+m, n)$ **do**
    $B_n[k,j] \leftarrow \tilde{\mathbf{d}}_k[j-k+1]$ (Store upper banded part)
  end for
  Step 4: Update matrices
  $[J, K, E_s, X_s, Y_s, Z_s] \leftarrow$ UpdateMatrices($J, K, E_s, X_s, Y_s, Z_s, \bar{\mathbf{k}}_k, \mathbf{b}_k, \tau_k$) (Algorithm 4.0.4)
end for
Final step:
$B_n[n,n] \leftarrow A_{n-1}[n,n]$
return $B_n + \mathrm{tril}(UV_n^\top, -1) + \mathrm{triu}(W_n\tilde{S}^\top, 1)$ and $\boldsymbol{\tau}$.

Algorithm 4.0.2 FormHouseholderVector

Input $A_{k-1} \in \mathbb{R}^{(n-k+1)\times(n-k+1)}$: the current trailing submatrix at step k, expressible as the remaining block of the original matrix plus accumulated structured perturbations, i.e., $A_{k-1} = A[k:n, k:n] + P_{k-1}$ where $P_{k-1} \in \mathcal{P}(A[k:n, k:n])$; $U_k \in \mathbb{R}^{(n-k+1)\times r}$: $U[k:n,:]$;$V_k \in \mathbb{R}^{(n-k+1)\times r}$: $V[k:n,:]$; ℓ: lower bandwidth.
Output $\bar{\mathbf{k}} \in \mathbb{R}^r$, $\mathbf{b} \in \mathbb{R}^{n-k+1}$, $\tau \in \mathbb{R}$ s.t. $\mathbf{y} := \mathbf{e}_1 + \tilde{U}_k\bar{\mathbf{k}} + \mathbf{b}$ is the Householder vector where $\tilde{U}_k \in \mathbb{R}^{(n-k+1)\times r}$ with $\tilde{U}_k[1,:] = \mathbf{0}$ and $\tilde{U}_k[2:n-k+1,:] = U[k+1:n,:]$. $I - \tau\mathbf{y}\mathbf{y}^\top$ is the Householder transformation. (Note that the expression for $\mathbf{y}$ here differs from that in Eq. (7); both forms are valid, and it is straightforward to convert between them.)
$\tilde{\mathbf{b}}_k \leftarrow$ Eq. (2), computed using the entries available within the input A_{k-1}.
$\alpha_k \leftarrow$ Eq. (3), computed entirely from A_{k-1}.
$\mathbf{b} \leftarrow$ Eq. (4)
$\bar{\mathbf{k}} \leftarrow$ Eq. (5)
$\mathbf{b}$ is only nonzero in entries 2 through $\min(\ell+1, n-k+1)$.
$\tau \leftarrow 2/\mathbf{y}^\top\mathbf{y}$
return $\bar{\mathbf{k}}$, $\mathbf{b}$, τ

Algorithm 4.0.3 ComputeRowUpdate

Input Current submatrix A_{k-1}, matrices $U, \tilde{S}$, a $\mathbf{k}_k$, and scaling vector $\mathbf{b}_k$, τ_k.
Output semiseparable part $\tilde{\mathbf{w}} \in \mathbb{R}^{r+p}$ and banded part $\tilde{\mathbf{d}} \in \mathbb{R}^{n-k+1}$
$\tilde{\mathbf{w}}^\top \leftarrow$ Eq.(9)
$\tilde{\mathbf{d}}^\top \leftarrow$ Eq.(10)
$\tilde{\mathbf{d}}$ is only nonzero in entries 1 through $\min(\ell + m + 1, n - k + 1)$.
return $\tilde{\mathbf{w}}, \tilde{\mathbf{d}}$

Algorithm 4.0.4 UpdateMatrices

Input J, K, E_s, X_s, Y_s, Z_s: current parameter matrices; $\bar{\mathbf{k}}_k$, $\mathbf{b}_k$, τ_k.
Ensure: Updated J, K, E_s, X_s, Y_s, Z_s
Compute updates using formulas from Lemma 4 proof:
$J_{\text{new}} \leftarrow$Eq. (11)
$K_{\text{new}} \leftarrow$Eq. (12)
$E_s^{\text{new}} \leftarrow$Eq. (13)
$X_s^{\text{new}} \leftarrow$Eq. (14)
$Y_s^{\text{new}} \leftarrow$Eq. (15)
$Z_s^{\text{new}} \leftarrow$Eq. (16)
return $J_{\text{new}}, K_{\text{new}}, E_s^{\text{new}}, X_s^{\text{new}}, Y_s^{\text{new}}, Z_s^{\text{new}}$

Complexity Analysis

The algorithm runs for $n - 1$ steps. The cost per step can be expressed as a polynomial in terms of r, p, ℓ, and m. Since these are constants independent of n, the total complexity is $O(n)$ as $n \to \infty$. The memory footprint is also $O(n)$, as we store only the generators and banded components.

Remark 4.1. *To maintain the $O(1)$ per-step complexity in Algorithm 4.0.1, two key quantities must be computed efficiently during the Householder updates:*

Inner product matrix $U[k:n,:]^\top U[k:n,:]$: The computation of many intermediate vectors requires evaluating expressions like $U[k:n,:]^\top \mathbf{y}_k$, which involves $U[k:n,:]^\top U[k:n,:]$. So we precompute a lookup table:

$$UU\_lookup[k] = U[k:n,:]^\top U[k:n,:] \quad \text{for } k = 1, \ldots, n$$

This can be computed in $O(nr^2)$ time via a backward accumulation.

Partial sum $\sum_{t=1}^{j} U[t,:]W[t,:]^\top$: The update of the upper triangular part for factor matrix requires this sum(Eq.6). We precompute:

$$UV\_lookup[j] = \sum_{t=1}^{j} U[t,:]W[t,:]^\top \quad \text{for } j = 1, \ldots, n-1$$

This is computed in $O(nrp)$ time via forward accumulation.

Both precomputations require $O(n)$ total time and enable $O(1)$ access to the required quantities at each step of the factorization, thus preserving the overall $O(n)$ complexity.

4.1. Fast Solver for BPS Matrices

Theorem 1 not only enables an efficient QR factorization but also facilitates a complete direct solver for linear systems of the form $A\mathbf{x} = \mathbf{b}$, where A is a BPS matrix. The solver consists of two phases after the QR factorization $A = QR$: 1. Application of $Q^\top$ to the right-hand side vector $\mathbf{b}$ to form $\mathbf{c} = Q^\top \mathbf{b}$. 2. Solution of the upper triangular system $R\mathbf{x} = \mathbf{c}$ via backward substitution.

We now present $O(n)$ algorithms for both phases, leveraging the structured representation of the factorization output by Algorithm 4.0.1.

4.1.1. Fast Application of $Q^\top$

The orthogonal matrix Q is represented as a product of Householder transformations:

$$Q = (I - \tau_1 \mathbf{y}_1 \mathbf{y}_1^\top)(I - \tau_2 \mathbf{y}_2 \mathbf{y}_2^\top) \cdots (I - \tau_{n-1} \mathbf{y}_{n-1} \mathbf{y}_{n-1}^\top).$$

Applying $Q^\top$ to a vector $\mathbf{b}$ thus requires computing:

$$Q^\top \mathbf{b} = (I - \tau_{n-1} \mathbf{y}_{n-1} \mathbf{y}_{n-1}^\top) \cdots (I - \tau_2 \mathbf{y}_2 \mathbf{y}_2^\top)(I - \tau_1 \mathbf{y}_1 \mathbf{y}_1^\top)\mathbf{b}.$$

The Householder vectors $\mathbf{y}_k$ are stored in the factor matrix F according to the normalization convention established in Section 2:

$$\mathbf{y}_k[j] = \begin{cases} 0, & j < k \\ 1, & j = k \\ F[j,k], & j > k \end{cases} \quad \text{for } k = 1, \ldots, n-1.$$

From Theorem 1, the factor matrix F admits the BPS representation:

$$F = B_n + \mathrm{tril}(U V_n^\top, -1) + \mathrm{triu}(W_n \tilde{S}^\top, 1),$$

where $U, V_n \in \mathbb{R}^{n \times r}$, $W_n, \tilde{S} \in \mathbb{R}^{n \times (r+p)}$, and B_n is banded with lower bandwidth ℓ and upper bandwidth $\ell + m$.

This structure implies that each Householder vector $\mathbf{y}_k$ can be expressed as:

$$\mathbf{y}_k = \mathbf{e}_1 + \tilde{U}_k V_n[k, :] + \mathbf{d}_k, \tag{17}$$

where:

$\tilde{U}_k \in \mathbb{R}^{n \times r}$ satisfies $\tilde{U}_k[1 : k, :] = \mathbf{0}$ and $\tilde{U}_k[k+1 : n, :] = U[k+1 : n, :]$,

and $\mathbf{d}_k \in \mathbb{R}^n$ is non-zero only in entries $k+1$ through $\min(k+\ell, n)$, with $\mathbf{d}_k[j] = B_n[j,k]$ for $j = k+1, \ldots, \min(k+\ell, n)$.

Algorithm 4.1.1 exploits this structure to compute $Q^\top \mathbf{b}$ in $O(n)$ operations by maintaining a compressed representation of the intermediate vectors throughout the transformation process.

Theorem 2. *Algorithm 4.1.1 computes $\mathbf{c} = Q^\top \mathbf{b}$ in $O(n)$ operations.*

Proof. The proof proceeds by induction on the transformation steps. Let $\mathbf{b}^{(0)} = \mathbf{b}$ and assume that after $k-1$ steps, the algorithm maintains the representation:

$$\mathbf{b}^{(k-1)} = \mathbf{b} + \tilde{U}_{k-1} \mathbf{h}^{(k-1)} + \sum_{j=1}^{r} \mathbf{o}_j^{(k-1)} + \sum_{j=1}^{\ell+1} \mathbf{g}_j^{(k-1)},$$

where the superscripts on $\mathbf{h}$, $\mathbf{o}$, and $\mathbf{g}$ denote the state after the $(k-1)$-th iteration.

The k-th Householder transformation gives:

$$\mathbf{b}^{(k)} = (I - \tau_k \mathbf{y}_k \mathbf{y}_k^\top)\mathbf{b}^{(k-1)} = \mathbf{b}^{(k-1)} - \tau_k (\mathbf{y}_k^\top \mathbf{b}^{(k-1)}) \mathbf{y}_k.$$

Substituting the structured form of $\mathbf{y}_k$ from (17) and the inductive representation:

$$\begin{aligned} \mathbf{b}^{(k)} &= \mathbf{b} + \tilde{U}_{k-1} \mathbf{h}^{(k-1)} + \sum_{j=1}^{r} \mathbf{o}_j^{(k-1)} + \sum_{j=1}^{\ell+1} \mathbf{g}_j^{(k-1)} \\ &\quad - \tau_k c (\mathbf{e}_1 + \tilde{U}_{k-1} \bar{\mathbf{v}}_k + \mathbf{d}_k) \\ &= \mathbf{b} + \tilde{U}_k (\mathbf{h}^{(k-1)} - \tau_k c \bar{\mathbf{v}}_k) \\ &\quad + \left(\sum_{j=1}^{r} \mathbf{o}_j^{(k-1)} + (\tilde{U}_{k-1} - \tilde{U}_k) \mathbf{h}^{(k-1)} \right) \end{aligned}$$

Algorithm 4.1.1 Fast Application of $Q^\top$ to a Vector

Input: Factor matrix $F \in \mathrm{BPS}^{n\times n}_{(r,r+p),(\ell,\ell+m)}$, scaling vector $\boldsymbol{\tau} = [\tau_1, \ldots, \tau_{n-1}, 0]^\top \in \mathbb{R}^n$, right-hand side $\mathbf{b} \in \mathbb{R}^n$.
Output: $\mathbf{c} = Q^\top \mathbf{b} \in \mathbb{R}^n$
Initialize:
$O \leftarrow \mathbf{0}_{n\times r}$: Storage for accumulated low-rank updates
$G \leftarrow \mathbf{0}_{n\times(\ell+1)}$: Storage for banded component updates
$\mathbf{h} \leftarrow \mathbf{0}_r$: Accumulator for semiseparable component
Let $\mathbf{o}_i$ denote the i-th column of O
Let $\mathbf{g}_i$ denote the i-th column of G
Express initial vector: $\mathbf{b}^{(0)} = \mathbf{b} + \tilde{U}_0\mathbf{h} + \sum_{j=1}^{r} \mathbf{o}_j + \sum_{j=1}^{l+1} \mathbf{g}_j$
for $k = 1$ to $n-1$ **do**
Compute inner product: $c \leftarrow \mathbf{y}_k^\top \mathbf{b}^{(k-1)}$ (exploit BPS structure of $\mathbf{y}_k$ and precompute some lookup tables for for $O(1)$ computation)
Update low-rank storage: $O[k,:] \leftarrow U[k,:] \odot \mathbf{h}$ (element-wise multiplication)
Update semiseparable accumulator: $\mathbf{h} \leftarrow \mathbf{h} - \tau_k c V_n[k,:]$
Update banded component:
$G[k,1] \leftarrow -\tau_k c$ (diagonal contribution)
for $t = 1$ to $\min(\ell, n-k)$ **do**
$G[k+t, t+1] \leftarrow -\tau_k c \cdot B_n[k+t,k]$ (subdiagonal contributions)
end for
Current representation: $\mathbf{b}^{(k)} = \mathbf{b} + \tilde{U}_k\mathbf{h} + \sum_{j=1}^{r} \mathbf{o}_j + \sum_{j=1}^{\ell+1} \mathbf{g}_j$
end for
Compute final result explicitly: $\mathbf{c} \leftarrow \mathbf{b} + \tilde{U}_{n-1}\mathbf{h} + \sum_{j=1}^{r} \mathbf{o}_j + \sum_{j=1}^{l+1} \mathbf{g}_j$
return c

$$+ \left(\mathbf{g}_1^{(k-1)} - \tau_k c \mathbf{e}_1\right) + \left(\sum_{j=2}^{\ell+1} \mathbf{g}_j^{(k-1)} - \tau_k c \mathbf{d}_k\right).$$

The algorithm updates precisely these components:
$\mathbf{h}^{(k)} = \mathbf{h}^{(k-1)} - \tau_k c \bar{\mathbf{v}}_k$,
$O[k,:] = U_n[k,:] \odot \mathbf{h}^{(k-1)}$ captures $(\tilde{U}_{k-1} - \tilde{U}_k)\mathbf{h}^{(k-1)}$,
Banded updates in G capture the remaining terms.

Thus, the representation is maintained correctly throughout all $n-1$ steps. Each step requires $O(1)$ operations due to the constant-bounded parameters r, p, ℓ, m, yielding overall $O(n)$ complexity. □

4.1.2. Fast Backward Substitution

After computing $\mathbf{c} = Q^\top \mathbf{b}$, we solve the upper triangular system $R\mathbf{x} = \mathbf{c}$, where $R = \mathrm{triu}(F)$ inherits the BPS structure of F. Specifically, the upper triangular part of F satisfies:

$$R = B_R + \mathrm{triu}(W_n \tilde{S}^\top, 1),$$

where $B_R = \mathrm{triu}(B_n)$ is the upper triangular part of the banded component of the factor matrix, maintaining upper bandwidth $\ell + m$.

Algorithm 4.1.2, which is equivalent to the one introduced in Olver and Townsend (2013), exploits this structure to perform backward substitution in $O(n)$ operations by maintaining a running sum for the semiseparable contributions.

Theorem 3. *Algorithm 4.1.2 solves $R\mathbf{x} = \mathbf{c}$ in $O(n)$ operations.*

Proof. For completeness we include the proof from Olver and Townsend (2013). The algorithm implements standard backward substitution while exploiting the structure of R. For each index j from n down to 1, the equation:

$$R[j,j]x_j + \sum_{k=j+1}^{n} R[j,k]x_k = c_j$$

Algorithm 4.1.2 Fast Backward Substitution for Structured R

Input: Upper triangular matrix $R = \mathrm{triu}(F)$ in structured form; transformed right-hand side $\mathbf{c} \in \mathbb{R}^n$
Output: Solution $\mathbf{x} \in \mathbb{R}^n$ satisfying $R\mathbf{x} = \mathbf{c}$
Initialize:
  $\mathbf{x} \leftarrow \mathbf{0}_n$: solution vector
  $\mathbf{s} \leftarrow \mathbf{0}_{r+p}$: Accumulator for semiseparable contributions
for $j = n$ down to 1 **do**
  Initialize accumulated sum: sum $\leftarrow 0$
  Add semiseparable contribution: sum $\leftarrow$ sum $+ W_n[j, :]^\top \mathbf{s}$
  Add banded contributions:
  for $k = j + 1$ to $\min(j + \ell + m, n)$ **do**
    sum $\leftarrow$ sum $+ B_n[j, k]\mathbf{x}[k]$
  end for
  Solve for x_j: $\mathbf{x}[j] \leftarrow (\mathbf{c}[j] - \text{sum})/B_n[j, j]$
  Update semiseparable accumulator: $\mathbf{s} \leftarrow \mathbf{s} + \tilde{S}[j, :]^\top \mathbf{x}[j]$
end for
return x

is solved for x_j.

The key insight is that the off-diagonal entries $R[j, k]$ for $k > j$ can be decomposed as:

$$R[j, k] = B_n[j, k] + W_n[j, :]^\top \tilde{S}[k, :].$$

The banded contributions $B_R[j, k]$ are non-zero only for $k = j+1, \ldots, \min(j+l+m, n)$, requiring $O(1)$ operations per row. The semiseparable contributions are accumulated in the vector $\mathbf{s}$, which stores:

$$\mathbf{s} = \sum_{k=j+1}^{n} x_k \tilde{S}[k, :].$$

At step j, the product $W_n[j, :]^\top \mathbf{s}$ thus captures all semiseparable contributions from previously computed solution components. After computing x_j, the accumulator is updated to include its contribution.

Each iteration requires $O(1)$ operations, yielding overall $O(n)$ complexity. The correctness follows by induction from $j = n$ down to 1. □

4.1.3. Overall Solver Complexity

Combining the QR factorization (Algorithm 4.0.1), the fast application of $Q^\top$ (Algorithm 4.1.1), and the fast backward substitution (Algorithm 4.1.2) yields a complete direct solver for BPS linear systems with $O(n)$ complexity.

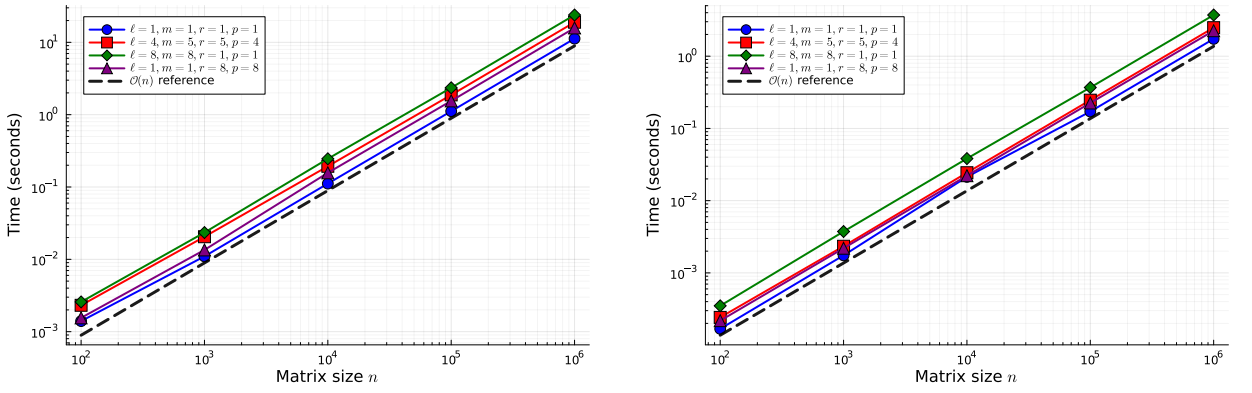
Corollary 1. *For a BPS matrix $A \in \mathbb{R}^{n\times n}$ with constant-bounded ranks and bandwidths, the linear system $A\mathbf{x} = \mathbf{b}$ can be solved in $O(n)$ operations using the QR-based approach.*

5. Numerical results

To validate the theoretical complexity and demonstrate the practical efficiency of our proposed algorithms, we implemented the fast QR factorization, the complete linear solver, and the fast RQ product computation for symmetric BPS matrices in Julia. The implementation is publicly available in the SemiseparableMatrices.jl package Chen, Olver et al. (2026), providing an open-source resource for the scientific computing community. Computations were carried out on a MacBook Air equipped with an Apple M2 chip (8-core CPU, 8 GB RAM), without GPU acceleration or access to external computing resources.

5.1. Linear Complexity Verification

To verify that the theoretical $O(n)$ complexity holds for matrices with different structural characteristics, we test four parameter sets (ℓ, m, r, p) representing different balances between the banded and semiseparable components.



(a) QR factorization phase.

(b) solution phase.

Figure 1: Log-log plots of execution times versus matrix size n for four parameter sets (ℓ, m, r, p), demonstrating optimal linear complexity. (a) Execution time for the structured QR factorization; (b) Execution time for the solve phase (applying $Q^\top$ and backward substitution). The dashed lines have a slope of 1, indicating perfect linear scaling $O(n)$.

Minimal case: $(1, 1, 1, 1)$; balanced case: $(4, 5, 5, 4)$; band-dominated case: $(8, 8, 1, 1)$; semiseparable-dominated case: $(1, 1, 8, 8)$.

Figure 1 demonstrates the linear time complexity of our complete QR solver for BPS linear systems. The execution time scales as $O(n)$ across five orders of magnitude, from $n = 100$ to $n = 10^6$, for four different parameter configurations. All curves align with the reference line of slope 1, confirming the analysis in Section 4.

While the computational time scales as $O(n)$, the underlying constant factor depends polynomially on the structural parameters r, p, ℓ, and m. Specifically, the cost per step is dominated by updating the perturbation space $\mathcal{P}(A)$, leading to an overall complexity of $O(n \cdot (r^2 + rp + r(\ell + m) + \ell p + \ell(\ell + m)))$ for the complete solver. This strict parameter dependence accounts for the vertical offsets among the curves in Figure 1 and Figure 5: configurations with larger semiseparable ranks or wider bandwidths exhibit an elevated constant overhead, while maintaining the identical linear scaling slope of 1.

5.2. Numerical Stability of the QR Solver

The numerical stability of a QR-based linear solver is crucial for its practical utility. Figure 2 demonstrates the excellent stability properties of our structured QR solver across a wide range of matrix sizes and structural parameters. We measure the accuracy through the maximum norm of the residual vector:

$$\max |A\mathbf{x} - \mathbf{b}|,$$

where $\mathbf{x}$ is the solution computed by our solver for a randomly generated right-hand side $\mathbf{b}$.

To isolate the numerical behavior of the QR factorization itself, all test matrices are constructed to be well-conditioned; more precisely, they are diagonally dominant. This ensures that the observed residuals are not influenced by ill-conditioning of the linear system, but instead reflect only the numerical errors introduced by the factorization and solution process.

For all parameter configurations and across six orders of magnitude in n (from $n = 10$ to $n = 10^6$), the computed residuals remain very small. The growth of the error is gradual: for smaller matrices, it is near machine precision ($\sim 10^{-16}$), while for the largest matrices, the maximum error corresponds to approximately 14 digits of accuracy. Even for the most challenging case (semiseparable-dominated with $r = p = 8$) and the largest $n = 10^6$, the algorithm preserves a maximum residual error of less than 5×10^{-15}, demonstrating its robustness and reliable numerical stability across a wide range of matrix structures and sizes.

Remark 5.1. *Measuring the accuracy of large-scale QR factorization presents a methodological challenge. Traditional approaches like comparing the factor matrix $\tilde{F}$ generated by our structured QR algorithm against the factor matrix*

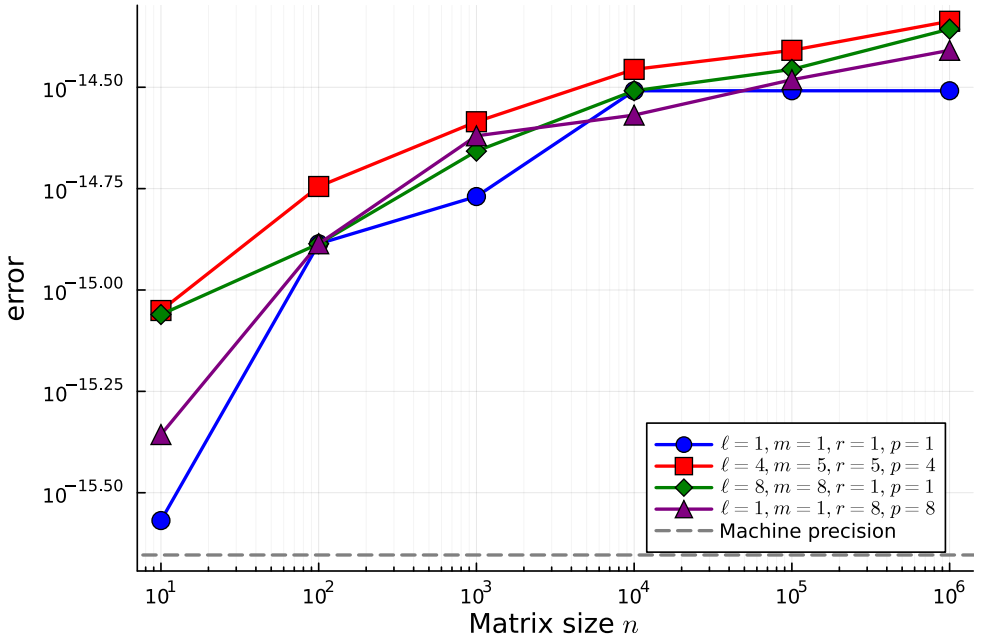


Figure 2: Log-log plot of the maximum residual $\max |A\mathbf{x} - \mathbf{b}|$ versus matrix size n for four parameter sets (ℓ, m, r, p). Here $\mathbf{x}$ is computed by our complete QR solver (QR factorization followed by application of $Q^\top$ and backward substitution). The dashed gray line indicates the machine precision $\epsilon_{\text{machine}} \approx 2.22 \times 10^{-16}$.

F_{dense} produced by a standard dense QR routine, or evaluating the reconstruction error $\|\tilde{Q}\tilde{R} - A\|$ where $\tilde{Q}$, $\tilde{R}$ are extracted from $\tilde{F}$ would require $O(n^3)$ operations to compute the dense equivalent matrices and therefore becomes infeasible for $n > 10^4$.

We overcome this limitation by leveraging the structure of BPS matrices: after obtaining the solution $\mathbf{x}$ via our $O(n)$ solver, we compute the residual $A\mathbf{x} - \mathbf{b}$ using the $O(n)$ matrix-vector multiplication algorithm for BPS matrices Chandrasekaran et al. (2002). This approach allows us to verify accuracy for matrices as large as $n = 10^6$ while maintaining overall $O(n)$ computational cost.

5.3. Comparison with HODLR QR

We compare our fast QR factorization against the state-of-the-art HODLR (Hierarchically Off-Diagonal Low-Rank) QR implementation from the hm-toolbox Massei et al. (2020). Hm-toolbox provides efficient MATLAB routines for various structured matrices, including HODLR and HSS matrices, and represents one of the most mature implementations for hierarchical matrix computations.

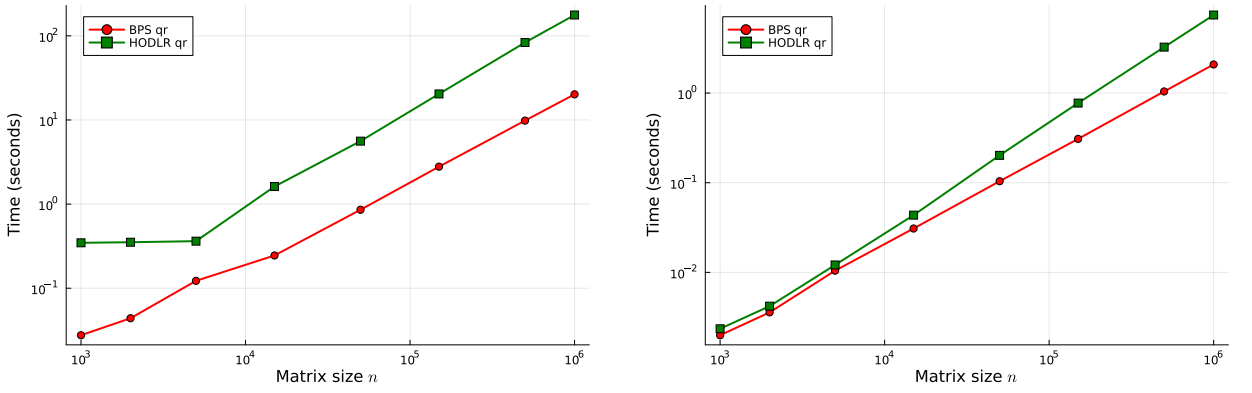
Crucially, while general HODLR-based QR factorization algorithms typically incur an asymptotic computational complexity of $O(n \log n)$ due to their hierarchical partitioning schemes, our proposed specialized algorithm achieves an optimal linear complexity of $O(n)$ by directly preserving and tracking the BPS structure throughout the reflections.

Figure 3 compares the execution times of QR factorization for BPS matrices obtained using the two approaches. Our algorithm consistently exhibits superior performance as the matrix size increases. This deterministic scaling advantage arises from its explicit exploitation of the underlying BPS structure, which altogether avoids the $O(n \log n)$ hierarchical overhead associated with general low-rank approximations.

Moreover, the performance gap widens with increasing n, confirming that our method is particularly well suited for large-scale problems. For $n = 1{,}000{,}000$, our implementation achieves an approximate 9× speedup over the HODLR approach, highlighting the practical benefits of the proposed specialized algorithm.

5.4. LAPACK Compatibility and Hybrid Solver Performance

A key practical benefit of our Householder-based formulation is its native compatibility with standard LAPACK storage formats. Because our factored matrix F and the scaling vector $\boldsymbol{\tau}$ conform strictly to standard LAPACK layouts,



(a) QR factorization phase.

(b) solution phase.

Figure 3: Log-log performance comparisons between our proposed fast BPS QR algorithm and the HODLR QR implementation from Massei et al. (2020) (with parameters $\ell = 4$, $m = 5$, $r = 2$, $p = 3$). Panel (a) illustrates the execution times for the structured QR factorization phase; Panel (b) presents the execution times for the solve phase (applying $Q^\top$ and backward substitution).

we can implement a highly efficient hybrid solver strategy. We benchmark three distinct strategies for solving $Ax = b$ with n ranging from 500 to 10,000 under fixed parameters $\ell = 4, m = 5, r = 2$, and $p = 3$. The first strategy is the BPS-only solver, which is our structured $O(n)$ solver using specialized in-place routines. The second is the LAPACK-only solver, which is a fully dense standard $O(n^3)$ solver. The third is the BPS-LAPACK hybrid solver, which first computes the fast $O(n)$ BPS factorization, converts the resulting factored BPS matrix into a standard dense format, and then applies LAPACK's highly optimized, BLAS-3 level $O(n^2)$ dense back-solvers. The performance scaling curves are presented in Figure 4.

The benchmarks show clear trade-offs. While the dense solver scales poorly and takes 16.93 seconds at $n = 10,000$, our BPS-only solver scales linearly and finishes in just 1.01 seconds. For smaller systems ($n \leq 1,000$), the BPS-only solver is slightly slower because applying the structured $Q^\top$ and R^{-1} in Julia requires looping over many small, individual Householder vectors. The hybrid solver resolves this overhead. By combining our fast $O(n)$ factorization with a conversion to a dense matrix, it hands the back-solve to LAPACK's highly optimized $O(n^2)$ solver, which runs on contiguous memory without loop overhead. Consequently, the hybrid solver runs in just 3.47 seconds at $n = 10,000$—a fivefold speedup over the pure dense solver (16.93 seconds)—while matching the top speed of the dense solver at smaller scales.

6. Conclusions

In this paper, we have established a fundamental theoretical result for BPS matrices and developed efficient algorithms based on this foundation. Our main contribution is the explicit proof that the standard QR factorization of a BPS matrix via Householder reflections preserves the BPS structure, with precisely characterized ranks and bandwidths in the resulting factor matrix. This theoretical insight enabled the design of a complete, Householder-based $O(n)$ direct solver for BPS linear systems that is natively compatible with LAPACK storage layouts, comprising: (1) a structure-preserving QR factorization algorithm (Algorithm 4.0.1); (2) an efficient $O(n)$ application of $Q^\top$ (Algorithm 4.1.1); and (3) a fast backward substitution routine (Algorithm 4.1.2).

Furthermore, for symmetric BPS matrices, we have shown that the reverse RQ product also maintains the BPS structure. This establishes the algebraic closure of the BPS matrix class under symmetric orthogonal similarity transformations and provides a crucial computational building block for more complex matrix algebra. This theoretical guarantee led to the development of another $O(n)$ algorithm (Algorithm A.2.1) for computing the RQ product without explicitly forming the dense orthogonal matrix Q.

The numerical experiments confirm the linear scaling and accuracy of our approach, while demonstrating significant performance advantages over existing HODLR-based methods. Additionally, our solver's native LAPACK

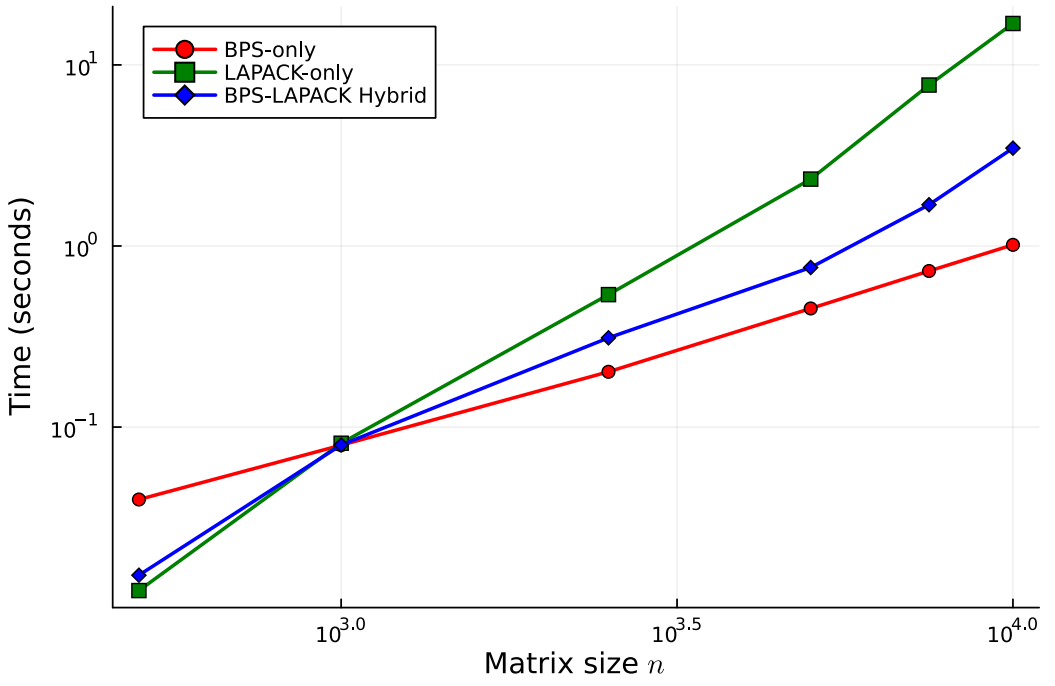


Figure 4: Performance scaling comparison for solving $Ax = b$ using the pure BPS $O(n)$ solver, the standard dense LAPACK $O(n^3)$ solver, and the hybrid BPS-LAPACK $O(n^2)$ solver.

Move this info to the previous factorisation/solve plot. Add both Unblocked and CompactWY

compatibility allows for a highly efficient hybrid approach that bridges the gap between structured and dense solvers. Our implementation in the SemiseparableMatrices.jl package provides the scientific computing community with efficient, open-source tools for working with this important class of structured matrices.

Future Work

A compelling extension involves applying our methodology to specific blocked banded matrices arising in *hp*-FEM Knook, Olver and Papadopoulos (2025). These have optimal complexity so-called reverse Cholesky factorizations (Cholesky from the bottom right instead of the top left) for positive definite problems. One of our motivations for the present work is developing an optimal complexity QL factorization for these special block banded matrices. The key challenge is generalizing our framework to block BPS matrices while maintaining $O(N)$ complexity. The primary difficulty lies in applying Householder transformations from one block to subsequent blocks in $O(n)$ time (where n is block size and N the total size), rather than $O(n^3)$. While our current framework doesn't directly apply, the core insight of structure preservation provides a promising foundation for this challenging extension.

A. Fast RQ Computation for Symmetric BPS Matrices

The fast QR factorization developed in the previous section provides an efficient direct solver for BPS linear systems. Beyond system inversion, studying the reverse RQ product yields fundamental insights into the algebraic closure of the BPS matrix class under orthogonal similarity transformations, as $RQ = Q^\top AQ$ when A is symmetric. Analyzing this product also establishes a key computational building block for more complex chain-structured orthogonal matrix decompositions. For symmetric BPS matrices, we show that the RQ product strictly preserves the BPS structure, leading to the design of a linear-complexity algorithm for its fast computation.

A.1. Structure of RQ

Consider a symmetric BPS matrix A of the form

$$A = B_s + \mathrm{tril}(UV^\top, -1) + \mathrm{triu}(VU^\top, 1), \tag{18}$$

where B_s is a symmetric banded matrix with lower and upper bandwidth ℓ, and $U, V \in \mathbb{R}^{n\times r}$ generate the lower and upper semiseparable parts of rank r. Let $A = QR$ be its QR factorization. Since $RQ = (Q^{-1}A)Q = Q^\top AQ$ and A is symmetric, RQ is also symmetric.

To describe the structure of intermediate matrices in the computation of RQ, we introduce definitions analogous to those in Section 3 but tailored for the symmetric case and the R factor.

Definition 6. *Denote the set of symmetric banded matrices with both lower and upper-bandwidth ℓ as $\mathrm{SBM}_\ell^{n\times n} \subset \mathbb{R}^{n\times n}$. In particular, $B \in \mathrm{SBM}_\ell^{n\times n} \subset \mathbb{R}^{n\times n}$ if $B = (b_{kj})_{k,j=1}^n \in \mathbb{R}^{n\times n}$ is a symmetric banded matrix satisfying $b_{kj} = 0$ for $k - j > \ell$ or $j - k > \ell$.*

Definition 7. *Denote the set of symmetric banded-plus-semiseparable matrices (SBPS) with lower and upper-semisepable rank r and lower and upper-bandwidth ℓ as $\mathrm{SBPS}_{r,\ell}^{n\times n} \subset \mathbb{R}^{n\times n}$. In particular, $A \in \mathrm{SBPS}_{r,\ell}^{n\times n} \subset \mathbb{R}^{n\times n}$ if*

$$A = B_s + \mathrm{tril}(UV^\top, -1) + \mathrm{triu}(VU^\top, 1)$$

for $U, V \in \mathbb{R}^{n\times r}$ and $B \in \mathrm{SBM}_\ell^{n\times n}$.

Following the QR factorization of $A \in \mathrm{SBPS}_{r,\ell}^{n\times n}$, we let R be the upper triangular factor and $I - \tau_k \mathbf{y}_k \mathbf{y}_k^\top$ be the k-th Householder reflector. We define the sequence of matrices $\{R_k\}$ such that $R_0 = R$ and $R_k = R_{k-1}(I - \tau_k \mathbf{y}_k \mathbf{y}_k^\top)$ for $k = 1, \ldots, n-1$, leading to $R_{n-1} = RQ$. The structural properties of R_k are governed by its trailing columns, which we formally describe using the following definition:

Definition 8. *Given an upper triangular matrix $R \in \mathbb{R}^{n\times n}$ and $U \in \mathbb{R}^{n\times r}$, define the linear space:*

$$\mathcal{H}(R; U) := \Bigg\{ RU\Omega U^\top + \Phi U^\top + RU\Psi + \Lambda : \\ \Omega \in \mathbb{R}^{r\times r}, \Phi \in \mathcal{S}_{\ell\times r}^{n\times r}, \Psi \in \mathcal{S}_{r\times \ell}^{r\times n}, \Lambda \in \mathcal{S}_{\ell\times\ell}^{n\times n} \Bigg\} \subset \mathbb{R}^{n\times n}.$$

For convenience we also define the nonzero portions of these matrices as $\Phi_s := \Phi[1 : \min(\ell, n), :]$, $\Psi_s := \Psi[: , 1 : \min(\ell, n)]$, and $\Lambda_s := \Lambda[1 : \min(\ell, n), 1 : \min(\ell, n)]$.

We characterize the structural evolution of the intermediate matrices R_k as follows:

Definition 9. *We say $R_k \in \mathrm{SBPSR}_k(A)$ if for*

$$\tilde{U} := RU \in \mathbb{R}^{n\times r},$$

there exist $B_k \in \mathrm{SBM}_\ell^{n\times n}$ and $V_k \in \mathbb{R}^{n\times r}$ so that the first k columns below the diagonal satisfy the conditions of the SBPS format:

$$\mathrm{tril}(R_k[:, 1:k]) = \mathrm{tril}\left((B_k + \mathrm{tril}(\tilde{U}V_k^\top, -1) + \mathrm{triu}(V_k\tilde{U}^\top, 1))[:, 1:k]\right)$$

whilst the bottom right block is a perturbed upper triangular matrix:

$$R_k[k+1:n, k+1:n] = R[k+1:n, k+1:n] + H_k,$$

where $H_k \in \mathcal{H}(A[k+1:n, k+1:n]; U[k+1:n, :])$.

We will show that the $R_k \in \mathrm{SBPSR}_k(A)$ by induction. As a preliminary step we note that the perturbation structure of the bottom right is preserved under truncation:

Lemma 5. *If $H \in \mathcal{H}(R;U)$ then $H[k:n,k:n] \in \mathcal{H}(R[k:n,k:n];U[k:n,:])$.*

Proof. We show for $k=2$ with the other cases following by induction. We write:

$$\begin{aligned}H[2:n,2:n] =&(RU)[2:n,:]\Omega U[2:n,:]^\top + \Phi[2:n,:]U[2:n,:]^\top\\ &+ (RU)[2:n,:]\Psi[:,2:n] + \Lambda[2:n,2:n].\end{aligned}$$

Since R is upper triangular, it follows that

$$(RU)[2:n,:] = R[2:n,2:n]U[2:n,:],$$

and hence

$$\begin{aligned}H[2:n,2:n] =&R[2:n,2:n]U[2:n,:]\Omega U[2:n,:]^\top + \Phi[2:n,:]U[2:n,:]^\top\\ &+ R[2:n,2:n]U[2:n,:]\Psi[:,2:n] + \Lambda[2:n,2:n]\\ \in&\mathcal{H}(R[2:n,k:n];U[2:n,:]).\end{aligned}$$

□

The structural form of the first k columns of R_k below the diagonal is characterized by the following lemma:

Lemma 6. *For $1 \le k \le n-1$, suppose $R_{k-1} \in \mathrm{SBPSR}_{k-1}(A)$. Then there exists B_k and V_k such that*

$$\mathrm{tril}(R_k[:,1:k]) = \mathrm{tril}\left((B_k + \mathrm{tril}(\tilde{U}V_k^\top,-1) + \mathrm{triu}(V_k\tilde{U}^\top,1))[:,1:k]\right)$$

Proof. We begin by defining the symmetric matrix B_k through its lower triangular part:

$$\mathrm{tril}(B_k[:,1:k-1]) := \mathrm{tril}(B_{k-1}[:,1:k-1]).$$

Similarly, for V_k we set

$$V_k[1:k-1,:] = V_{k-1}[1:k-1,:].$$

It remains to specify $B_k[k:n,k]$ and $V_k[k,:]$. Note that the entries $B_k[k,k:n]$ are implicitly determined by the symmetry of B_k, and any other entries not involved in the current or previous k steps of the induction may be assigned arbitrarily at this stage.

Since $R_{k-1} \in \mathrm{SBPSR}_{k-1}(A)$, we may write

$$R_{k-1}[k:n,k:n] = R[k:n,k:n] + H_{k-1}$$

where

$$\begin{aligned}H_{k-1} :=&R[k:n,k:n]U[k:n,:]\Omega_k U[k:n,:]^\top + \Phi_k U[k:n,:]^\top\\ &+ R[k:n,k:n]U[k:n,:]\Psi_k + \Lambda_k\end{aligned}$$

with $\Omega_k \in \mathbb{R}^{r\times r}$, $\Phi_k \in \mathcal{S}_{\ell\times r}^{(n-k+1)\times r}$, $\Psi_k \in \mathcal{S}_{r\times\ell}^{r\times(n-k+1)}$, and $\Lambda_k \in \mathcal{S}_{\ell\times\ell}^{(n-k+1)\times(n-k+1)}$.

We now consider applying the k-th Householder reflection $I - \tau_k \mathbf{y}_k\mathbf{y}_k^\top$ to

$$\begin{aligned}R_{k-1}[k:n,k:n] =&R[k:n,k:n] + R[k:n,k:n]U[k:n,:](\Omega_k U[k:n,:]^\top + \Psi_k)\\ &+ \Phi_k U[k:n,:]^\top + \Lambda_k.\end{aligned}$$

Since R is upper triangular, we have

$$R[k:n,k:n]U[k:n,:] = (RU)[k:n,:],$$

which allows us to rewrite

$$R_{k-1}[k:n,k:n] = R[k:n,k:n] + (RU)[k:n,:]\underbrace{(\Omega_k U[k:n,:]^\top + \Psi_k)}_{:=L_1\in\mathbb{R}^{r\times(n-k+1)}} + \underbrace{\Phi_k U[k:n,:]^\top + \Lambda_k}_{:=L_2\in\mathcal{S}_{\ell\times r}^{(n-k+1)\times r}}.$$

Focusing on the first column, we obtain

$$R_{k-1}[k:n,k:n]\mathbf{e}_1 = \mathbf{t}_1 + (RU)[k:n,:]\underbrace{(\Omega_k U[k,:] + \Psi_k \mathbf{e}_1)}_{:=\mathbf{r}_1\in\mathbb{R}^r}$$

where

$$\mathbf{t}_1 := R[k:n,k] + \Phi_k U[k,:] + \Lambda_k[:,1] \in \mathcal{S}_\ell^{n-k+1}.$$

The Householder vector $\mathbf{y}_k$ can be expressed as $\mathbf{y}_k = \hat{\mathbf{b}}_k + U[k:n,:]\bar{\mathbf{k}}_k$ for some $\bar{\mathbf{k}}_k \in \mathbb{R}^r$ from Eq.(7), and we compute

$$\begin{aligned} R[k:n,k:n]\mathbf{y}_k &= \mathbf{t}_2 + R[k:n,k:n]U[k:n,:]\bar{\mathbf{k}}_k \\ &= \mathbf{t}_2 + (RU)[k:n,:]\bar{\mathbf{k}}_k \end{aligned}$$

with

$$\mathbf{t}_2 := R[k:n,k:n]\hat{\mathbf{b}}_k \in \mathcal{S}_{\ell+1}^{n-k+1}.$$

Hence

$$R_{k-1}[k:n,k:n]\mathbf{y}_k = \underbrace{\mathbf{t}_2 + L_2\mathbf{y}_k}_{=:\mathbf{t}_3\in\mathcal{S}_{\ell+1}^{n-k+1}} + (RU)[k:n,:]\underbrace{(\bar{\mathbf{k}}_k + L_1\mathbf{y}_k)}_{=:\mathbf{r}_2\in\mathbb{R}^r}.$$

Therefore, letting $\delta_k := -\tau_k \mathbf{y}_k^\top \mathbf{e}_1$, applying the Householder reflection yields

$$R_{k-1}[k:n,k:n](I - \tau_k\mathbf{y}_k\mathbf{y}_k^\top)\mathbf{e}_1 = (\mathbf{t}_1 + \delta_k\mathbf{t}_2) + RU[k:n,:](\mathbf{r}_1 + \delta_k\mathbf{r}_2)$$

which leads to the definitions

$$V_k[k,:] := \mathbf{r}_1 + \delta_k\mathbf{r}_2 \in \mathbb{R}^r, \tag{19}$$

$$B_k[k:n,k] := \mathbf{t}_1 + \delta_k\mathbf{t}_2 \in \mathcal{S}_{\ell+1}^{n-k+1}. \tag{20}$$

□

The following lemma completes the inductive step by demonstrating that the trailing submatrix of R_k preserves the perturbed structure defined in the space $\mathcal{H}$:

Lemma 7. *For $1 \le k \le n-1$, suppose $R_{k-1} \in \mathrm{SBPSR}_{k-1}(A)$. Then $R_k \in \mathrm{SBPSR}_k(A)$.*

Proof. We have already established the desired structure for all entries except the bottom-right block. It therefore remains to show that

$$\begin{aligned} R_k[k+1:n,k+1:n] &:= \left(R_{k-1}[k:n,k:n](I - \tau_k\mathbf{y}_k\mathbf{y}_k^\top)\right)[2:n-k+1,2:n-k+1] \\ &\in R[k+1:n,k+1:n] + \mathcal{H}(R[k+1:n,k+1:n];U[k+1:n,:]). \end{aligned}$$

By Lemma 5, since $R_{k-1}[k:n,k:n] \in R[k:n,k:n] + \mathcal{H}(R[k:n,k:n]; U[k:n,:])$, it follows that

$$R_{k-1}[k+1:n,k+1:n] \in R[k+1:n,k+1:n] + \mathcal{H}(R[k+1:n,k+1:n]; U[k+1:n,:]).$$

More explicitly, we may write

$$\begin{aligned} R_{k-1}[k+1:n,k+1:n] = R[k+1:n,k+1:n] \\ &+ R[k+1:n,k+1:n]U[k+1:n,:]\Omega_k U[k+1:n,:]^\top \\ &+ \Phi_k[2:n-k+1,:]U[k+1:n,:]^\top \\ &+ R[k+1:n,k+1:n]U[k+1:n,:]\Psi_k[:,2:n-k+1] \\ &+ \Lambda_k[2:n-k+1,2:n-k+1]. \end{aligned}$$

It thus remains to analyze the term $R_{k-1}[k:n,k:n]\mathbf{y}_k\mathbf{y}_k^\top$:

$$\begin{aligned} R_{k-1}[k:n,k:n]\mathbf{y}_k\mathbf{y}_k^\top &= \big(\mathbf{t}_3 + (RU)[k:n,:]\mathbf{r}_2\big)(\hat{\mathbf{b}}_k^\top + \bar{\mathbf{k}}_k^\top U[k:n,:]^\top) \\ &= \underbrace{\mathbf{t}_3\hat{\mathbf{b}}_k^\top}_{\in S^{(n-k+1)\times(n-k+1)}_{(\ell+1)\times(\ell+1)}} + \underbrace{\mathbf{t}_3\bar{\mathbf{k}}_k^\top}_{\in S^{(n-k+1)\times r}_{(\ell+1)\times r}} U[k:n,:]^\top \\ &\quad + (RU)[k:n,:]\underbrace{\mathbf{r}_2\hat{\mathbf{b}}_k^\top}_{\in S^{r\times(n-k+1)}_{r\times(\ell+1)}} \\ &\quad + (RU)[k:n,:]\underbrace{\mathbf{r}_2\bar{\mathbf{k}}_k^\top}_{\in\mathbb{R}^{r\times r}} U[k:n,:]^\top. \end{aligned}$$

Using

$$\big((RU)[k:n,:]\big)[2:n-k+1,:] = R[k+1:n,k+1:n]U[k+1:n,:],$$

we can express

$$\begin{aligned} R_k[k+1:n,k+1:n] &= R_{k-1}[k+1:n,k+1:n] \\ &\quad - \tau_k\big(R_{k-1}[k:n,k:n]\mathbf{y}_k\mathbf{y}_k^\top\big)[2:n-k+1,2:n-k+1] \\ &= R[k+1:n,k+1:n] \\ &\quad + R[k+1:n,k+1:n]U[k+1:n,:]\Omega_{k+1}U[k+1:n,:]^\top \\ &\quad + \Phi_{k+1}U[k+1:n,:]^\top \\ &\quad + R[k+1:n,k+1:n]U[k+1:n,:]\Psi_{k+1} + \Lambda_{k+1} \end{aligned}$$

where

$$\Omega_{k+1} := \Omega_k - \tau_k\mathbf{r}_2\bar{\mathbf{k}}_k^\top \in \mathbb{R}^{r\times r}, \tag{21}$$

$$\Phi_{k+1} := \Phi_k[2:n-k+1,:] - \tau_k\mathbf{t}_3[2:n-k+1]\bar{\mathbf{k}}_k^\top \in S^{(n-k)\times r}_{\ell\times r}, \tag{22}$$

$$\Psi_{k+1} := \Psi_k[:,2:n-k+1] - \tau_k\mathbf{r}_2\hat{\mathbf{b}}_k^\top[2:n-k+1] \in S^{r\times(n-k)}_{r\times\ell},, \tag{23}$$

$$\Lambda_{k+1} := \Lambda_k[2:n-k+1,2:n-k+1] - \tau_k\mathbf{t}_3[2:n-k+1]\hat{\mathbf{b}}_k^\top[2:n-k+1] \tag{24}$$
$$\in S^{(n-k)\times(n-k)}_{\ell\times\ell}.$$

□

Combining these inductive results, we arrive at the central theorem of this section: the RQ product preserves the SBPS structure of A, including both the semiseparable rank r and the bandwidth ℓ:

Theorem 4. *After the QR factorization of a SBPS matrix $A \in \mathrm{SBPS}^{n\times n}_{r,\ell}$, RQ also $\in \mathrm{SBPS}^{n\times n}_{r,\ell}$.*

Proof. It is easy to see that $R_0 := R \in \mathrm{SBPSR}_0(A)$ (with the parameter matrices Ω, Φ, Ψ, and Λ all $\mathbf{0}$). Applying Lemma 6 and 7, it follows by induction that

$$\mathrm{tril}(R_{n-1}[:, 1 : n-1]) = \mathrm{tril}\left(B_{n-1} + \mathrm{tril}(\tilde{U}V_{n-1}^\top, -1) + \mathrm{triu}(V_{n-1}\tilde{U}^\top, 1)\right).$$

Now define B_n by setting $B_n[n, n] = R_{n-1}[n, n]$ and letting it coincide with B_{n-1} elsewhere, and set $V_n := V_{n-1}$. Then

$$\mathrm{tril}(R_{n-1}) = \mathrm{tril}\left(B_n + \mathrm{tril}(\tilde{U}V_n^\top, -1) + \mathrm{triu}(V_n\tilde{U}^\top, 1)\right).$$

Since $QR = R_{n-1}$ and both sides are symmetric, it follows that

$$QR = B_n + \mathrm{tril}(\tilde{U}V_n^\top, -1) + \mathrm{triu}(V_n\tilde{U}^\top, 1).$$

□

A.2. Fast RQ Multiplication

Theorem 4 leads directly to a fast algorithm for computing the RQ product without explicitly forming the dense orthogonal matrix Q.

Algorithm A.2.1 Fast RQ computation for Symmetric BPS Matrices

Input: A symmetric BPS matrix A given by its generators: symmetric banded B_s (bandwidth ℓ), and $U, V \in \mathbb{R}^{n\times r}$ satisfying $A = B_s + \mathrm{tril}(UV^\top, -1) + \mathrm{triu}(VU^\top, 1)$.
Output: The RQ product in structured form: symmetric banded matrix B_n, and low-rank generators $\tilde{U}$ and V_n.

1. **Compute fast QR factorization:**
Run Algorithm 4.0.1 on A to obtain the structured factor matrix F and coefficients $\boldsymbol{\tau}$. Extract the structured representation of the R factor and all Householder vectors $\mathbf{y}_k$ (with parameters $\bar{\mathbf{k}}_k$, $\mathbf{b}_k$).
Compute and store $\tilde{U} = RU$ using the structured R in $O(n)$.

2. **Initialize parameters:**
Set $\Omega \leftarrow \mathbf{0}_{r\times r}$, $\Phi_s \leftarrow \mathbf{0}_{\min(\ell,n)\times r}$, $\Psi_s \leftarrow \mathbf{0}_{r\times\min(\ell,n)}$, $\Lambda_s \leftarrow \mathbf{0}_{\min(\ell,n)\times\min(\ell,n)}$.
Initialize $V_n \leftarrow \mathbf{0}_{n\times r}$.

3. **Forward recursion (apply Q from the right):**
for $k = 1$ to $n-1$ **do**
a. Retrieve the Householder parameters $\bar{\mathbf{k}}_k$ and $\mathbf{b}_k$ for step k.
b. Update $\Omega \leftarrow$ Eq. (21), $\Phi_s \leftarrow$ Eq. (22), $\Psi_s \leftarrow$ Eq. (23), and $\Lambda_s \leftarrow$ Eq. (24).
c. Set $V_n[k, :] \leftarrow$ Eq.(19) and $B_n[k : \min(k+\ell, n), k] \leftarrow$ Eq.(20).
end for

4. **Final State**
Set $B_n[n, n] \leftarrow R_{n-1}[n, n]$

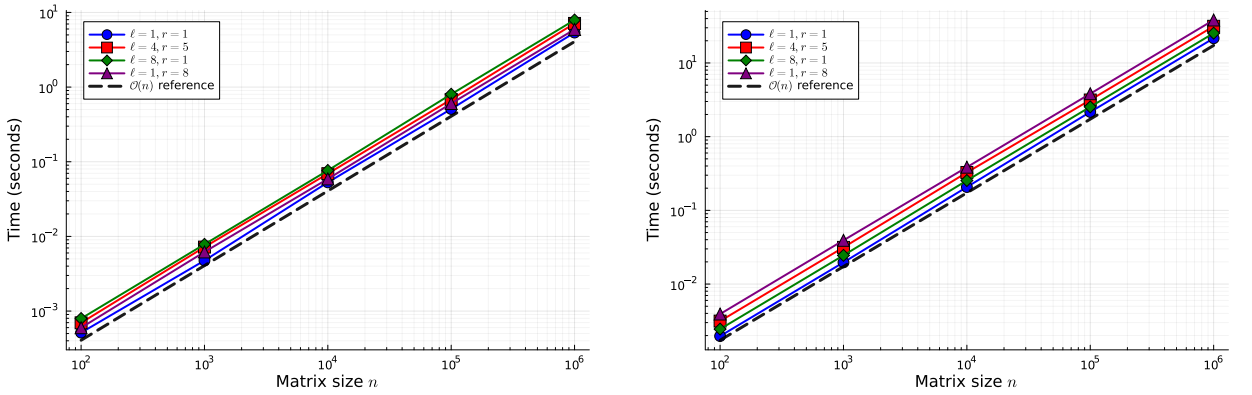
5. **Return** $B_n, \tilde{U}, V_n$.

Complexity analysis. Step 1, the QR factorization requires $O(n)$ operations. The computation of $\tilde{U} = RU$ can be performed in $O(nr)$ time due to the structure of R. The forward recursion in Step 3 performs a constant amount of work per iteration, as all matrix operations either involve matrices whose dimensions (e.g., $r\times r$, $r\times\ell$, $\ell\times\ell$) are independent of n, or involve $U[k : n, :]^\top U[k : n, :]$, which can be evaluated in $O(1)$ time after an $O(n)$ time preprocessing step to build a lookup table. Therefore, Algorithm A.2.1 runs in $O(n)$ time and uses $O(n)$ storage.

Remark A.1. *A subtle but crucial point in Algorithm A.2.1 is that during the forward recursion, we track only the evolving structure of the submatrix $\boldsymbol{R}_j[j+1:n, j+1:n]$, even though the entire matrix $\boldsymbol{R}_j[:, j+1:n]$ is being modified. More precisely, after applying the first j Householder transformations from the right, the first j columns of $\boldsymbol{R}_j$ have reached their final state and will not change in subsequent steps. However, the first j rows of $\boldsymbol{R}_j$ (in columns $j+1:n$) are still subject to modification by later transformations. Nevertheless, because the final product $\boldsymbol{RQ}$ is known to be symmetric, these pending row entries are not independent: they must eventually equal the corresponding entries in the already-fixed columns. This allows the algorithm to safely disregard the explicit updating of these rows and focus solely on the lower-right square submatrix, which is the only part whose future evolution is not predetermined.*

Remark A.2 (Perturbation and Shift Invariance)**.** *The structural preservation results extend directly to algebraic shifts applied to the matrix. Given a scalar shift $\mu \in \mathbb{R}$, the shifted matrix $\boldsymbol{A}-\mu\boldsymbol{I} = (\boldsymbol{B}_s-\mu\boldsymbol{I})+\mathrm{tril}(\boldsymbol{UV}^\top,-1)+\mathrm{triu}(\boldsymbol{VU}^\top,1)$ remains a symmetric BPS matrix, requiring only a standard diagonal correction to the banded part $\boldsymbol{B}$. The resulting factored product can then be adjusted back to obtain $\boldsymbol{RQ}+\mu\boldsymbol{I}$, which naturally inherits the identical BPS formatting boundaries. Hence, the algebraic closure of the BPS class under these linear translations remains intact and can be computed entirely within $O(n)$ time.*

Figure 5 confirms the $O(n)$ complexity of computing the $\boldsymbol{RQ}$ product for symmetric BPS matrices. The four parameter sets exhibit linear scaling, with all curves scaling parallel to the $O(n)$ reference line, thereby validating the algebraic and complexity analysis presented in this Appendix.



(a) Isolated $\boldsymbol{RQ}$ product loop.

(b) Combined $\boldsymbol{QR}$ and $\boldsymbol{RQ}$.

Figure 5: Log-log plots of computation times for symmetric BPS matrices versus n across four distinct parameter sets (ℓ, r). The left panel (a) illustrates the isolated $\boldsymbol{RQ}$ computation cost, while the right panel (b) presents the cumulative execution time encompassing both the complete structured $\boldsymbol{QR}$ factorization and the subsequent $\boldsymbol{RQ}$ product. The dashed reference lines possess a slope of 1, indicating strict $O(n)$ linear scaling for both components.

References

Anderson, E., Bai, Z., Bischof, C., Blackford, L.S., Demmel, J., Dongarra, J., Du Croz, J., Greenbaum, A., Hammarling, S., McKenney, A., Sorensen, D., 1999. LAPACK Users' Guide. Third ed., Society for Industrial and Applied Mathematics.

Chandrasekaran, S., Dewilde, P., Gu, M., Lyons, W., Pals, T., 2007. A fast solver for HSS representations via sparse matrices. SIAM Journal on Matrix Analysis and Applications 29, 67–81.

Chandrasekaran, S., Dewilde, P., Gu, M., Pals, T., van der Veen, A.J., 2002. Fast stable solver for sequentially semi-separable linear systems of equations, in: International Conference on High-Performance Computing, Springer. pp. 545–554.

Chandrasekaran, S., Gu, M., 2003. Fast and stable algorithms for banded plus semiseparable systems of linear equations. SIAM Journal on Matrix Analysis and Applications 25, 373–384.

Chandrasekaran, S., Gu, M., Pals, T., 2006. A fast ULV decomposition solver for hierarchically semiseparable representations. SIAM Journal on Matrix Analysis and Applications 28, 603–622.

Chen, T., Olver, S., et al., 2026. SemiseparableMatrices.jl. URL: https://github.com/JuliaLinearAlgebra/SemiseparableMatrices.jl.

Delvaux, S., Van Barel, M., 2006a. Rank structures preserved by the QR-algorithm: the singular case. Journal of Computational and Applied Mathematics 189, 157–178.

Delvaux, S., Van Barel, M., 2006b. Structures preserved by the QR-algorithm. Journal of Computational and Applied Mathematics 187, 29–40.

Delvaux, S., Van Barel, M., 2008a. A Givens-weight representation for rank structured matrices. SIAM Journal on Matrix Analysis and Applications 29, 1147–1170.

Delvaux, S., Van Barel, M., 2008b. A QR-based solver for rank structured matrices. SIAM Journal on Matrix Analysis and Applications 30, 464–490.

Eidelman, Y., Gohberg, I., 1997. Inversion formulas and linear complexity algorithm for diagonal plus semiseparable matrices. Computers & Mathematics with Applications 33, 69–79.

Eidelman, Y., Gohberg, I., Haimovici, I., 2014. Separable type representations of matrices and fast algorithms. volume 1. Springer.

Eidelman, Y., Gohberg, I., Olshevsky, V., 2005. The QR iteration method for Hermitian quasiseparable matrices of an arbitrary order. Linear Algebra and its Applications 404, 305–324.

Fasino, D., 2005. Rational Krylov matrices and QR steps on Hermitian diagonal-plus-semiseparable matrices. Numerical linear algebra with applications 12, 743–754.

Iserles, A., 2025. Stable spectral methods for time-dependent problems and the preservation of structure. Foundations of Computational Mathematics 25, 683–723.

Knook, K., Olver, S., Papadopoulos, I., 2025. Quasi-optimal complexity hp-FEM for Poisson on a rectangle. IMA Journal of Numerical Analysis , draf102.

Massei, S., Robol, L., Kressner, D., 2020. hm-toolbox: MATLAB software for HODLR and HSS matrices. SIAM Journal on Scientific Computing 42, C43–C68.

Mastronardi, N., Chandrasekaran, S., Van Huffel, S., 2001. Fast and stable two-way algorithm for diagonal plus semi-separable systems of linear equations. Numerical linear algebra with applications 8, 7–12.

Olver, S., Townsend, A., 2013. A fast and well-conditioned spectral method. siam REVIEW 55, 462–489.

Rózsa, P., Bevilacqua, R., Romani, F., Favati, P., 1991. On band matrices and their inverses. Linear Algebra and Its Applications 150, 287–295.

Van Camp, E., Mastronardi, N., Van Barel, M., 2004. Two fast algorithms for solving diagonal-plus-semiseparable linear systems. Journal of Computational and Applied Mathematics 164, 731–747.

Vandebril, R., Van Barel, M., . Manual for sspack: the semiseparable software package .

Vandebril, R., Van Barel, M., Mastronardi, N., 2005a. An implicit QR algorithm for symmetric semiseparable matrices. Numerical Linear Algebra with Applications 12, 625–658.

Vandebril, R., Van Barel, M., Mastronardi, N., 2005b. A note on the representation and definition of semiseparable matrices. Numerical Linear Algebra with Applications 12, 839–858.

Vandebril, R., Van Barel, M., Mastronardi, N., 2008a. Matrix computations and semiseparable matrices: linear systems. volume 1. JHU Press.

Vandebril, R., Van Barel, M., Mastronardi, N., 2008b. Rational QR-iteration without inversion. Numerische Mathematik 110, 561–575.

Xia, J., Chandrasekaran, S., Gu, M., Li, X.S., 2010. Fast algorithms for hierarchically semiseparable matrices. Numerical Linear Algebra with Applications 17, 953–976.